

Efficient Algorithms for Constructing Very Sparse Spanners and Emulators*

Michael Elkin^{†1} and Ofer Neiman^{‡1}

¹Department of Computer Science, Ben-Gurion University of the Negev, Beer-Sheva, Israel.
Email: {elkinm, neimano}@cs.bgu.ac.il

Abstract

Miller et al. [MPVX15] devised a distributed¹ algorithm in the CONGEST model, that given a parameter $k = 1, 2, \dots$, constructs an $O(k)$ -spanner of an input unweighted n -vertex graph with $O(n^{1+1/k})$ expected edges in $O(k)$ rounds of communication. In this paper we improve the result of [MPVX15], by showing a k -round distributed algorithm in the same model, that constructs a $(2k - 1)$ -spanner with $O(n^{1+1/k}/\epsilon)$ edges, with probability $1 - \epsilon$, for any $\epsilon > 0$. Moreover, when $k = \omega(\log n)$, our algorithm produces (still in k rounds) *ultra-sparse* spanners, i.e., spanners of size $n(1 + o(1))$, with probability $1 - o(1)$. To our knowledge, this is the first distributed algorithm in the CONGEST or in the PRAM models that constructs spanners or skeletons (i.e., connected spanning subgraphs) that sparse. Our algorithm can also be implemented in linear time in the standard centralized model, and for large k , it provides spanners that are sparser than any other spanner given by a known (near-)linear time algorithm.

We also devise improved bounds (and algorithms realizing these bounds) for $(1 + \epsilon, \beta)$ -spanners and emulators. In particular, we show that for any unweighted n -vertex graph and any $\epsilon > 0$, there exists a $(1 + \epsilon, (\frac{\log \log n}{\epsilon})^{\log \log n})$ -emulator with $O(n)$ edges. All previous constructions of $(1 + \epsilon, \beta)$ -spanners and emulators employ a superlinear number of edges, for all choices of parameters.

Finally, we provide some applications of our results to approximate shortest paths' computation in unweighted graphs.

1 Introduction

1.1 Setting, Definitions

We consider unweighted undirected n -vertex graphs $G = (V, E)$. For a parameter $\alpha \geq 1$, a subgraph $H = (V, E')$, $E' \subseteq E$, is called an α -spanner of G , if for every pair $u, v \in V$ of vertices, we have $d_H(u, v) \leq \alpha \cdot d_G(u, v)$. Here $d_G(u, v)$ (respectively, $d_H(u, v)$) stands for the distance between u and v in G (resp., in H). The parameter α is called the *stretch* of the spanner H . More generally, if for a pair of parameters $\alpha \geq 1$, $\beta \geq 0$, for every pair $u, v \in V$ of vertices, it holds that $d_H(u, v) \leq \alpha \cdot d_G(u, v) + \beta$, then the subgraph H is said to be an (α, β) -spanner of G . Particularly important is the case $\alpha = 1 + \epsilon$, for

*A preliminary version [EN17] of this paper appeared in SODA'17.

[†]This research was supported by the ISF grant No. (724/15).

[‡]Supported in part by ISF grant No. (523/12) and by BSF grant No. 2015813.

¹They actually showed a PRAM algorithm. The distributed algorithm with these properties is implicit in [MPVX15].

some small $\epsilon > 0$. Such spanners are called *near-additive*. If $H = (V, E'', \omega)$, where $\omega : E'' \rightarrow \mathbb{R}^+$, is not a subgraph of G , but nevertheless satisfies that for every pair $u, v \in V$ of original vertices, $d_G(u, v) \leq d_H(u, v) \leq (1 + \epsilon)d_G(u, v) + \beta$, then H is called a *near-additive β -emulator* of G , or a $(1 + \epsilon, \beta)$ -emulator of G .

Graph spanners have been introduced in [Awe85, PS89, PU89a], and have been intensively studied ever since [ADD⁺93, ABCP93, Coh99, ACIM99, DHZ00, BS03, Elk04, Elk07a, EZ06, TZ06, Woo06, Elk07b, Pet09, DGPV08, Pet10, BW15, MPVX15, AB16]. They were found useful for computing approximately shortest paths [ABCP93, Coh99, Elk04, EZ06], routing [PU89b], distance oracles and labeling schemes [Pel99, TZ05, EP15], synchronization [Awe85], and in other applications.

The simplest and most basic algorithm for computing a multiplicative α -spanner, for a parameter $\alpha \geq 1$, is the *greedy* algorithm [ADD⁺93]. The algorithm starts with an empty spanner, and examines the edges of the input graph $G = (V, E)$ one after another. It tests if there is a path in H of length at most α between the endpoints u and v of e . If it is not the case, the edge is inserted into the spanner. Otherwise the edge is dropped.

It is obvious that the algorithm produces an α -spanner H . Moreover, the spanner H has no cycles of length $\alpha + 1$ or less, i.e., the *girth* of H , denoted $g(H)$, satisfies $g(H) \geq \alpha + 2$. Denote $m = m(n, g)$ the maximum number of edges that a girth- g n -vertex graph may contain. It follows that $|H| \leq m(n, \alpha + 2)$. The function $m(n, g)$ is known to be at most $n^{1 + \frac{2}{g-2}}$, when $g \leq 2 \log_2 n$, and for larger g (i.e., for $m \leq 2n$), it is given by $m(n, g) \leq n(1 + (1 + o(1)) \frac{\ln(p+1)}{g})$, where $p = m - n$, [AHL02, BR10]. These bounds are called “Moore’s bounds for irregular graphs”, or shortly, (*generalized*) *Moore’s bounds*.

Any construction of multiplicative α -spanners for n -vertex graphs with at most $m'(n, \alpha + 2)$ edges implies an upper bound $m(n, \alpha + 2) \leq m'(n, \alpha + 2)$ for the function $m(n, g)$. (As running the construction on the extremal girth- $(\alpha + 2)$ n -vertex graph can eliminate no edge.) Hence the greedy algorithm produces multiplicative spanners with optimal tradeoff between stretch and number of edges. (See also [FS16].) However, the greedy algorithm is problematic from algorithmic perspective. In the centralized model of computation, the best-known implementation of it [RZ04] requires $O(\alpha \cdot n^{2+1/\alpha})$ time. Moreover, the greedy algorithm is inherently sequential, and as such, it is generally hard² to implement it in distributed and parallel models of computation.

In the distributed model [Pel00] we have processors residing in vertices of the graph. The processors communicate with their graph neighbors in synchronous rounds. In each round, messages of bounded length can be sent. (This is the assumption of the CONGEST model. In the LOCAL model, messages’ size is arbitrary.) The running time of an algorithm in this model is the number of rounds that it runs. By “parallel” model we mean here PRAM EREW model [Rei93], and we are interested in algorithms with small *running time* (aka *depth*) and *work* complexities. (The latter measures the overall number of operations performed by all processors.)

Dubhashi et al. [DMP⁺03] devised a distributed implementation of the greedy algorithm in the LOCAL model of distributed computation. Their algorithm runs in $O(\alpha \cdot \log^2 n)$ rounds, i.e., suboptimal by a factor of $\log^2 n$. Moreover, it collects graph topology to depth $O(\alpha)$, and conducts heavy local computations. To our knowledge, there is no distributed-CONGEST or PRAM implementation of the greedy algorithm known. There is also no known efficient³ centralized, distributed-CONGEST, or PRAM algorithm that constructs *ultra-sparse* spanners, i.e., spanners with $n + o(n)$ edges.

²In the sequel we discuss a distributed setting, specifically, the LOCAL model, in which a relatively efficient implementation of the greedy is known.

³By “efficient” centralized algorithm in this paper we mean an algorithm with running time close to $O(|E|)$. By efficient distributed or parallel algorithm we mean an algorithm that runs in polylogarithmic, or nearly-polylogarithmic, time.

In the distributed and parallel settings it is often enough to compute a sparse *skeleton* H of the input graph G , where a *skeleton* is a connected subgraph that spans all the vertices of G , i.e., the stretch requirement is dropped. Dubhashi et al. [DMP⁺03] devised a distributed-LOCAL algorithm that computes ultra-sparse skeletons of size $m(n, \alpha) \leq n + O(n \cdot \frac{\log n}{\alpha})$ in $O(\alpha)$ rounds. Like their algorithm for constructing spanners, this algorithm also collects topologies to depth $O(\alpha)$, and involves heavy local computations. To our knowledge, no efficient distributed-CONGEST or PRAM algorithm for computing *ultra*-sparse skeletons is currently known. In this paper we devise the first such algorithms.

1.2 Prior Work and Our Results

In the centralized model of computation the best-known efficient algorithm for constructing multiplicative spanners (for unweighted graphs) is due to Halperin and Zwick [HZ96]. Their deterministic algorithm, for an integer parameter $k \geq 1$, computes a $(2k - 1)$ -spanner with $n^{1+1/k} + n$ edges in $O(|E|)$ time. (Their result improved previous pioneering work by [PS89, Coh99].) Note that their bound on the number of edges is always at least $2n$, i.e., in the range $k = \Omega(\log n)$ it is very far from Moore's bound.

Our centralized randomized algorithm computes (with probability close to 1), a $(2k - 1)$ -spanner with $n \cdot (1 + O(\frac{\log n}{k}))$ edges in $O(|E|)$ time, whenever $k = \Omega(\log n)$. Note that when $k = \omega(\log n)$, the number of edges is $n(1 + o(1))$, i.e., in this range the algorithm computes an ultra-sparse spanner in $O(|E|)$ time. Moreover, whenever $k \leq n^{1-\epsilon}$, for any constant $\epsilon > 0$, up to a constant factor in the lower-order term, our bound matches Moore's bound. In fact, our algorithm and its analysis can be viewed as an alternative proof of (a slightly weaker version of) the generalized Moore's bound. Note that it is not the case for the greedy algorithm and its implementations [ADD⁺93, RZ04, DMP⁺03]: the analysis of these algorithms relies on Moore's bounds, but these algorithms cannot be used to derive them.

Another variant of our algorithm, which works for any $k \geq 2$, computes with high probability a $(2k - 1)$ -spanner with $n^{1+1/k}(1 + O(\frac{\log k}{k}))$ edges, in $\tilde{O}(k|E|)$ time.⁴ In particular, for the range $k \geq \frac{2 \ln n}{\ln \ln n}$ the number of edges in our spanner is $n^{1+1/k} + o(n)$, improving the result of [HZ96] (albeit with a somewhat worse running time for $\frac{2 \ln n}{\ln \ln n} \leq k \leq \log n$). Note that for any $k \geq 2$ we have $O(n^{1+1/k})$ edges.

Yet another variant of our algorithm computes a $(2k - 1)$ -spanner with $O(n^{1+1/k})$ edges, in expected $O(|E|)$ time.

In the distributed-CONGEST and PRAM models, efficient algorithms for computing linear-size spanners were given in [Pet09, MPVX15]. Specifically, [MPVX15] devised an $O(k)$ -round distributed-CONGEST randomized algorithm for constructing $O(k)$ -spanner (with high probability) with expected $O(n^{1+1/k})$ edges. In the PRAM model their algorithm has depth $O(k \log^* n)$ and work $O(|E|)$. There are also k -round distributed-CONGEST randomized algorithms for constructing $(2k - 1)$ -spanner with expected $O(k \cdot n^{1+1/k})$ edges [BS07, Elk07a]. It is known that at least k rounds are required for this task, under Erdős' girth conjecture [Elk07a, DGPV08].

Our randomized algorithm uses k rounds in the distributed-CONGEST model, and with probability at least $1 - \epsilon$ it constructs a $(2k - 1)$ -spanner with $O(n^{1+1/k}/\epsilon)$ edges (for any desired, possibly sub-constant, $\epsilon > 0$). In the PRAM model the depth and work complexities of our algorithm are the same as in [MPVX15]. Furthermore, when $k \geq \log n$ we can bound the number of edges by $n \cdot (1 + O(\frac{\log n}{\epsilon \cdot k}))$, again matching Moore's bound up to a constant factor in the lower-order term.

This result improves the previous state-of-the-art in the entire range of parameters. In particular, it is also the first efficient algorithm in the distributed-CONGEST or PRAM models that constructs an ultra-sparse skeleton. Specifically, in $\tilde{O}(\log n)$ time it computes an $\tilde{O}(\log n)$ -spanner with $n(1 + o(1))$ edges,

⁴As usual, $\tilde{O}(f(n))$ stands for $O(f(n)\text{polylog} f(n))$.

with probability $1 - o(1)$.

We also use our algorithm for unweighted graphs to devise an improved algorithm for *weighted* graphs as well. Specifically, our algorithm computes $(2k - 1)(1 + \epsilon)$ -spanner with $O(n^{1+1/k} \cdot (\log k)/\epsilon)$ edges, within expected $O(|E|)$ time. See Theorem 2, and the discussion that follows it, for further details.

1.3 Near-Additive Spanners and Emulators

It was shown in [EP04] that for any $\epsilon > 0$ and $\kappa = 1, 2, \dots$, and any (unweighted) n -vertex graph $G = (V, E)$, there exists a $(1 + \epsilon, \beta)$ -spanner with $O(\beta \cdot n^{1+1/\kappa})$ edges, where $\beta = \beta(\kappa, \epsilon) \leq O(\frac{\log \kappa}{\epsilon})^{\log \kappa}$. Additional algorithms for constructing such spanners were later given in [Elk04, EZ06, TZ06, DGPV08, Pet09, Pet10]. Abboud and Bodwin [AB16] showed that multiplicative error of $1 + \epsilon$ in [EP04]’s theorem cannot be eliminated, while still keeping a constant (i.e., independent of n) additive error β , and more recently [ABP17] showed that any such $(1 + \epsilon, \beta)$ -spanner of size $O(n^{1+1/\kappa-\delta})$, $\delta > 0$, has $\beta = \Omega\left(\frac{1}{\epsilon \cdot \log \kappa}\right)^{\log \kappa-1}$.

In the regime of constant κ , the bound of [EP04] remains the state-of-the-art. Pettie [Pet09] showed that one can construct a $(1 + \epsilon, \beta)$ -spanner with $O(n \log \log(\epsilon^{-1} \log \log n))$ edges and $\beta = O(\frac{\log \log n}{\epsilon})^{\log \log n}$. This result of [Pet09] is not efficient in the sense considered in this paper, i.e., no distributed or parallel implementations of it are known, and also no efficient (that is, roughly $O(|E|)$ -time) centralized algorithm computing it is known. Also, this result does not extend ([Pet16]) to a general tradeoff between β and the number of edges.

Improving upon previous results by [Elk05, EZ06], Pettie [Pet10] also devised an efficient distributed-CONGEST algorithm, that for a parameter $\rho > 0$, constructs in $\tilde{O}(n^\rho)$ rounds a $(1 + \epsilon, \beta)$ -spanner with $O(n^{1+1/\kappa}(\epsilon^{-1} \log \kappa)^\phi)$ edges and $\beta = O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log_\phi \kappa + 1/\rho}$, for $\phi = \frac{1+\sqrt{5}}{2}$ being the golden ratio.⁵ Independently and simultaneously to our work, [ABP17] showed that there exist $(1 + \epsilon, \beta)$ -spanners with $O((\epsilon^{-1} \log \kappa)^h \cdot \log \kappa \cdot n^{1+1/\kappa})$ edges and $\beta = O\left(\frac{\log \kappa}{\epsilon}\right)^{\log \kappa-2}$, where $h = \frac{(3/4)\kappa-1-\log \kappa}{\kappa} < 3/4$. This spanner has improved dependence on ϵ in the number of edges (at the cost of worse dependence on κ).

In this paper we improve all of the tradeoffs [EP04, Pet10] in the entire range of parameters. Specifically, for any $\epsilon > 0$, $\rho > 0$ and $\kappa = 1, 2, \dots$, $\frac{\log n}{\log(1/\epsilon) + \log \log \log n}$, our distributed-CONGEST algorithm constructs in $\tilde{O}(n^\rho)$ rounds a $(1 + \epsilon, \beta)$ -spanner with $O(n^{1+1/\kappa})$ edges and

$$\beta \leq O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log \kappa + 1/\rho}.$$

Our algorithm also admits efficient implementations in the streaming and standard models of computation, see Section 3. Our spanners are sparser and have polynomially smaller β than the previous best efficient constructions. They are even sparser than the state-of-the-art existential ones (with essentially the same β), with the following exceptions: whenever $\epsilon < 1/\log^3 \log n$ our result and that of [ABP17] are incomparable,⁶ and the spanner from [Pet09] that has $O(n \log^{(4)} n)$ edges, while ours never gets sparser than $O(n \log \log n)/\epsilon$. In the complementary range, $\epsilon > 1/\log^3 \log n$, our result is strictly stronger than that of [ABP17].

Moreover, a variant of our algorithm efficiently constructs very sparse $(1 + \epsilon, \beta)$ -emulators. In particular, we can obtain a *linear-size* $(1 + \epsilon, (\frac{\log \log n}{\epsilon})^{\log \log n})$ -emulator. (We stress that the number of edges does not depend even on ϵ .) All previous constructions of $(1 + \epsilon, \beta)$ -spanners or emulators employ a superlinear number of edges, for all choices of parameters.

⁵ In the range of $\kappa = o(\frac{\log n}{\log \log n})$, the result of [Pet10] is incomparable with [EP04], as spanners of [EP04] provide smaller β , while spanners of [Pet10] are slightly sparser.

⁶ The i -iterated logarithm is defined by $\log^{(i+1)} n = \log(\log^{(i)} n)$, for all $i \geq 0$, and $\log^{(0)} n = n$.

We use our new algorithms for constructing near-additive spanners and emulators to improve approximate shortest paths' algorithms, in the centralized and streaming models of computation. One notable result in this context is a streaming algorithm that for any constant $\epsilon > 0$ and any subset $S \subseteq V$ with $|S| = n^{\Omega(1)}$, computes $(1 + \epsilon)$ -approximate shortest paths for $S \times V$ within $O(|S|)$ passes over the stream, using $O(n^{1+\epsilon})$ space. See Section 4 for more details, and additional applications of our spanners.

1.4 Technical Overview

Linial and Saks [LS93] were the first to employ exponential random variables to build *network decompositions*, i.e., partitions of graphs into clusters of small diameter, which possess some useful properties. This technique was found useful for constructing padded partitions, hierarchically-separated trees, low-stretch spanning trees [Bar96, Bar98, Bar04, EEST05, ABN11, AN12] and spanners [Coh99, BS03, Elk07b]. In [LS93] every vertex v tosses a random variable r_v from an exponential distribution, and broadcasts to all vertices within distance r_v from v . Every vertex v joins the cluster of a vertex u with largest identity number, whose broadcast u heard.

Blelloch et al. [BGK⁺14] introduced a variant of this technique in which, roughly speaking, every vertex v starts to broadcast at time $-r_v$, and broadcasts indefinitely. A vertex x joins the cluster centered at a vertex v , whose broadcast reaches x first. They called the resulting partition “exponential start time clustering”, and it was demonstrated in [BGK⁺14, MPX13, EN16a] that this approach leads to very efficient distributed and parallel algorithms for constructing padded partitions and network decompositions.

Miller et al. [MPVX15] used this approach to devise an efficient parallel and distributed-CONGEST $O(k)$ -time algorithm for constructing $O(k)$ -spanner with $O(n^{1+1/k})$ edges. Specifically, they build the exponential time clustering, add the spanning trees of the clusters into the spanner, and then every vertex x adds into the spanner one edge (x, y) connecting x to every adjacent cluster C_y , $y \in C_y$.

The main property of the partition exploited by [MPVX15] in the analysis of their algorithm is that any unit-radius ball in the input graph G intersects just $O(n^{2/k})$ clusters, in expectation. Note also that their algorithm is doomed to use at least $n^{1+1/k} + (n - 1)$ edges, because it starts with inserting the spanning trees of all clusters (amounting to up to $n - 1$ edges), and then inserts the $O(n^{1+2/k})$ edges crossing between different clusters into the spanner. To get $O(n^{1+1/k})$ edges, one rescales $k' = 2k$.

In our algorithm we do not explicitly construct the exponential start time clustering. Rather we run the procedure that builds it, but every vertex x connects not just to the neighbor y through which x received its first broadcast message at time, say, t_y , but also to all neighbors z whose broadcast x received within time interval $[t_y, t_y + 1]$. We show that, in expectation, x connects to $n^{1/k}$ neighbors altogether, and not just to that many adjacent clusters. As a result we obtain both a sparser spanner, a smaller stretch, and a smaller running time. The stretch and running time are smaller roughly by a factor of 2 than in [MPVX15], because we do not need to consider unit balls, that have diameter 2. Rather we tackle individual edges (of length 1).

In the context of *weighted* graphs, [MPVX15] showed how their efficient algorithm for constructing sparse $(4k - 2)$ -spanners for unweighted graphs can be converted into an efficient algorithm that constructs $(16k - 8)$ -spanners with $O(n^{1+1/k} \log k)$ edges for weighted graphs. By using their scheme naively on top of our algorithm for unweighted graphs, one gets an efficient algorithm for computing $(4k - 2)(1 + \epsilon)$ -spanners with $O(n^{1+1/k} \cdot (\log k)/\epsilon)$ edges. Roughly speaking, the overhead of 2 in the stretch is because in the analysis of [MPVX15], every vertex contributes expected $O(n^{1/k})$ edges to the spanner on each of roughly $O(n^{1/k})$ phases of the algorithm in which it participates. By employing a more delicate probabilistic argument, we argue that in fact, the expected total contribution of every vertex in *all phases altogether* is $O(n^{1/k})$, rather than $O(n^{2/k})$. This enables us to eliminate another factor of 2 from the stretch bound. See Section 2.3 for details.

Our constructions of $(1 + \epsilon, \beta)$ -spanners and emulators follow the [EP04] *superclustering and interconnection* approach. One starts with a base partition $\mathcal{P}_0$. In [EP04] this was the partition of [Awe85, PS89, AP92], obtained via region-growing technique. Then every cluster $C \in \mathcal{P}_0$ that has “many” unclustered clusters of $\mathcal{P}_0$ “nearby”, creates a supercluster around it. The “many” and the “nearby” are determined by degree threshold $\deg_0$ and distance threshold δ_0 , respectively. Once the superclustering phase is over, the remaining unclustered clusters enter an interconnection phase, i.e., every pair of participating nearby clusters gets interconnected by a shortest path in the spanner. This completes one iteration of the process. The resulted superclustering $\mathcal{P}_1$ is the input for the next iteration of this process, which runs with different, carefully chosen thresholds $\deg_1$ and δ_1 . Such iterations continue until only very few clusters survive. The latter are interconnected without further superclustering.

One bottleneck in devising efficient distributed algorithm based on this approach is the base partition. Known algorithms for constructing a region-growing partition of [Awe85] require almost linear distributed time [DMZ06]. We demonstrate that one can bypass it completely, and start from the base partition $\mathcal{P}_0 = \{\{v\} \mid v \in V\}$. This requires some modification of the algorithm, and a more nuanced analysis. In addition, we show that the superclustering and interconnection steps themselves can be implemented efficiently. This part of the algorithm is based on our recent work on hopsets [EN16b], where we showed that [EP04] approach is extremely useful in that context as well, and that it can be made efficient.

1.5 Related Work

Efficient algorithms for constructing $(1 + \epsilon, \beta)$ -spanners were also devised in [Elk05, EZ06, TZ06, Pet09, Pet10]. These algorithms are based, however, on different approaches than that of the current paper. The latter is based on [EP04]. Specifically, the approach of [Elk05, EZ06] is based on [Coh99, Coh00] construction of pairwise covers and hopsets, i.e., the algorithm works top-down. It recurses in small clusters, and eliminates large ones. The approach of [TZ06, Pet09, Pet10] is based on [TZ05] collection of trees, used originally for distance oracles.

Streaming algorithms for constructing multiplicative spanners were given in [FKM⁺05, Elk07b, Bas08], and near-additive spanners in [Elk05, EZ06]. Spanners and emulators with sublinear error were given in [TZ06, Pet09]. Spanners with purely additive error and lower bounds concerning them were given in [ACIM99, EP04, BCE05, BKMP10, Che13, Woo06, BW15, AB16].

1.6 Organization

In Section 2 we present our algorithm for constructing multiplicative spanners and its analysis. In Section 2.3 we use this algorithm to provide improved spanners for weighted graphs as well. Our near-additive spanners and emulators are presented in Section 3.

2 Sparse Multiplicative Spanners and Skeletons

Let $G = (V, E)$ be a graph on n vertices, and let $k \geq 1$ be an integer. Let $c > 3$ be a parameter governing the success probability, and set $\beta = \ln(cn)/k$. Recall the exponential distribution with parameter β , denoted $\mathcal{EXP}(\beta)$, which has density

$$f(x) = \begin{cases} \beta \cdot e^{-\beta x} & x \geq 0 \\ 0 & \text{otherwise.} \end{cases}$$

Construction. Each vertex $u \in V$ samples a value r_u from $\mathcal{EX}\mathcal{P}(\beta)$, and broadcasts it to all vertices within distance k . Each vertex x that received a message originated at u , stores $m_u(x) = r_u - d_G(x, u)$, and also a neighbor $p_u(x)$ that lies on a shortest path from x to u (this neighbor sent x the message from u , breaking ties arbitrarily if there is more than one). Let $m(x) = \max_{u \in V} \{m_u(x)\}$, then for every $x \in V$ we add to the spanner H the set of edges

$$C(x) = \{(x, p_u(x)) : m_u(x) \geq m(x) - 1\} .$$

The following lemma is implicit in [MPVX15]. We provide a proof for completeness.

Lemma 1 ([MPVX15]). *Let $d_1 \leq \dots \leq d_n$ be arbitrary values and let $\delta_1, \dots, \delta_n$ be independent random variables sampled from $\mathcal{EX}\mathcal{P}(\beta)$. Define the random variables $M = \max_i \{\delta_i - d_i\}$ and $I = \{i : \delta_i - d_i \geq M - 1\}$. Then for any $1 \leq t \leq n$,*

$$\Pr[|I| \geq t] = (1 - e^{-\beta})^{t-1} .$$

Proof. Denote by $X^{(t)}$ the random variable which is the t -th largest among $\{\delta_i - d_i\}$. Then for any value $a \in \mathbb{R}$, if we condition on $X^{(t)} = a$, then the event $|I| \geq t$ is exactly the event that all the remaining $t - 1$ values $X^{(1)}, \dots, X^{(t-1)}$ are at least a and at most $a + 1$. Using the memoryless property of the exponential distribution and the independence of the $\{\delta_i\}$, we have that

$$\Pr[|I| \geq t \mid X^{(t)} = a] = (1 - e^{-\beta})^{t-1} .$$

Since this bound does not depend on the value of a , applying the law of total probability we conclude that

$$\Pr[|I| \geq t] = (1 - e^{-\beta})^{t-1} .$$

□

Using this lemma, we can bound the expected size of the spanner.

Lemma 2. *The expected size of H is at most $(cn)^{1/k} \cdot n$.*

Proof. Fix any $x \in V$, and we analyze $\mathbb{E}[|C(x)|]$. Note that the event $|C(x)| \geq t$ happens when there are at least t shifted random variables $r_u - d_G(u, x)$ that are within 1 of the maximum. By Lemma 1 this happens with probability at most $(1 - e^{-\beta})^{t-1}$ (we remark that if x did not hear at least t messages, then trivially $\Pr[|C(x)| \geq t] = 0$). We conclude that

$$\mathbb{E}[|C(x)|] = \sum_{t=1}^n \Pr[|C(x)| \geq t] \leq \sum_{t=0}^{\infty} (1 - e^{-\beta})^t = e^{\beta} = (cn)^{1/k} ,$$

and the lemma follows by linearity of expectation. □

We now argue about the stretch of the spanner.

Claim 3. *With probability at least $1 - 1/c$, it holds that $r_u < k$ for all $u \in V$.*

Proof. For any $u \in V$, $\Pr[r_u \geq k] = e^{-\beta k} = 1/(cn)$. By the union bound, $\Pr[\exists u, r_u \geq k] \leq 1/c$. □

Assume for now that the event of Claim 3 holds, i.e., that $r_u < k$ for all $u \in V$.

Corollary 4. *For any $x \in V$, if $u \in V$ is the vertex maximizing $m_u(x)$, then $d_G(u, x) < k$.*

Proof. First note that $m(x) \geq m_x(x) \geq 0$, and using Claim 3 we have $r_u < k$. So $0 \leq m(x) = m_u(x) = r_u - d_G(u, x) < k - d_G(u, x)$. $\square$

Claim 5. For any $u, x \in V$, if x adds an edge to $p_u(x)$, then there is a shortest path P between u and x that is fully contained in the spanner H .

Proof. We prove by induction on $d_G(u, x)$. In the base case $d_G(u, x) = 1$, then $p_u(x) = u$, so (x, u) is in the spanner. Assume that every vertex $y \in V$ with $d_G(u, y) = t - 1$ which added an edge to $p_u(y)$ has a shortest path to u in H , and we prove for x that has $d_G(u, x) = t$. We know that x added an edge to $y = p_u(x)$, which lies on a shortest path to u , and thus satisfies $d_G(u, y) = t - 1$. It remains to show that this y added an edge to $p_u(y)$. First we claim that

$$m(y) \leq m(x) + 1. \quad (1)$$

Seeking contradiction, assume that (1) does not hold, and let $v \in V$ be the vertex maximizing $m_v(y)$. By Corollary 4 we have $d_G(v, y) < k$, and thus $d_G(v, x) \leq k$. Hence x will hear the message of v . This means that $m_v(x) \geq m_v(y) - 1 = m(y) - 1 > m(x)$, which is a contradiction. This establishes (1). Now, since x added an edge to $y = p_u(x)$, by construction

$$m_u(x) \geq m(x) - 1. \quad (2)$$

We conclude that

$$m_u(y) = m_u(x) + 1 \stackrel{(2)}{\geq} m(x) - 1 + 1 \stackrel{(1)}{\geq} m(y) - 1,$$

so y indeed adds an edge to $p_u(y)$, and by the induction hypothesis we are done. $\square$

Lemma 6. The spanner H has stretch at most $2k - 1$.

Proof. Since H is a subgraph of G , it suffices to prove for any $(x, y) \in E$, that $d_H(x, y) \leq 2k - 1$. Let u be the vertex maximizing $m(x) = m_u(x)$, and w.l.o.g assume $m(x) \geq m(y)$. By Corollary 4 we have $d_G(u, x) \leq k - 1$, so $d_G(u, y) \leq k$, and y heard the message of u (which was sent to distance k). This implies that $m_u(y) \geq m_u(x) - 1 = m(x) - 1 \geq m(y) - 1$, so y adds the edge $(y, p_u(y))$ to the spanner. By applying Claim 5 on x and y , we see that both have shortest paths to u that are fully contained in H . Since $d_G(x, u) \leq k - 1$ and $d_G(y, u) \leq k$, these two paths provide stretch $2k - 1$ between x, y . $\square$

2.1 Main Theorem

We now state our main theorem, from which we will derive several interesting corollaries in various settings.

Theorem 1. For any unweighted graph $G = (V, E)$ on n vertices, any integer $k \geq 1$, $c > 3$ and $\delta > 0$, there is a randomized algorithm that with probability at least $(1 - 1/c) \cdot \delta / (1 + \delta)$ computes a spanner with stretch $2k - 1$ and number of edges at most

$$(1 + \delta) \cdot \frac{(cn)^{1+1/k}}{c - 1} - \delta(n - 1).$$

Proof. Let $\mathcal{Z}$ be the event that $\{\forall u \in V, r_u < k\}$. By Claim 3 we have $\Pr[\mathcal{Z}] \geq 1 - 1/c$. Note that conditioning on $\mathcal{Z}$, by Lemma 6 the algorithm produces a spanner $H = (V, E')$ with stretch $2k - 1$. In particular, it must have at least $n - 1$ edges. Let X be the random variable $|E'| - (n - 1)$, which conditioned on $\mathcal{Z}$ takes only nonnegative values. By Lemma 2 we have $\mathbb{E}[X] = (cn)^{1/k} \cdot n - (n - 1)$. We now argue

that conditioning on $\mathcal{Z}$ will not affect this expectation by much. Indeed, by the law of total probability, for any t , $\Pr[X = t] \geq \Pr[X = t \mid \mathcal{Z}] \cdot \Pr[\mathcal{Z}]$. Thus

$$\mathbb{E}[X \mid \mathcal{Z}] \leq \frac{\mathbb{E}[X]}{\Pr[\mathcal{Z}]} \leq \frac{c}{c-1} \cdot \left[(cn)^{1/k} \cdot n - (n-1) \right]. \quad (3)$$

By Markov inequality,

$$\Pr[X \geq (1+\delta)\mathbb{E}[X \mid \mathcal{Z}] \mid \mathcal{Z}] \leq \frac{1}{1+\delta}.$$

We conclude that

$$\Pr[(X < (1+\delta)\mathbb{E}[X \mid \mathcal{Z}]) \wedge \mathcal{Z}] = \Pr[X < (1+\delta)\mathbb{E}[X \mid \mathcal{Z}] \mid \mathcal{Z}] \cdot \Pr[\mathcal{Z}] \geq \left(1 - \frac{1}{c}\right) \cdot \frac{\delta}{1+\delta}.$$

If this indeed happens, then

$$\begin{aligned} |E'| &= X + n - 1 \\ &\stackrel{(3)}{\leq} (1+\delta) \cdot \frac{c}{c-1} \cdot \left[(cn)^{1/k} \cdot n - (n-1) \right] + n - 1 \\ &= (1+\delta) \cdot \frac{(cn)^{1+1/k} - (n-1)}{c-1} - \delta(n-1). \end{aligned}$$

□

2.1.1 Implementation Details

Distributed Model. It is straightforward to implement the algorithm in the LOCAL model of computation, it will take k rounds to execute it – in each round, every vertex sends to its neighbors all the messages it received so far. We claim that the algorithm can be implemented even when bandwidth is limited, i.e., in the CONGEST model. This will require a small variation: in each round, every vertex $v \in V$ will send to all its neighbors the message $(r_u, d_G(u, v))$ for the vertex u that currently maximizes $m_u(v) = r_u - d_G(u, v)$. We note that omitting all the other messages will not affect the algorithm, since if one such message would cause some neighbor of v to add an edge to v , then the message about u will suffice, as the latter has the largest $m_u(v)$ value. (Also recall that all vertices start their broadcast simultaneously, and do so for k rounds, so any omitted message could not have been sent to further distance than the message from u , which implies dropping it will have no effects on farther vertices as well.)

PRAM Model. In the parallel model of computation, we can use a variant of the construction that appeared in [MPX13, MPVX15]. Roughly speaking, vertex u will start its broadcast at time $k - \lceil r_u \rceil$, and every vertex x will send only the first message that arrives to it (which realizes $m(x)$). As argued in [MPVX15], the algorithm can be implemented in $O(k \log^* n)$ depth and $O(|E|)$ work.

Standard Centralized Model. Note that in the standard centralized model of computation, the running time is at most the work of the PRAM algorithm, which is $O(m)$. By taking constant c and δ , and repeating the algorithm until the first success (we can easily check the number of edges of the spanner and that all $r_u < k$), we get a spanner with stretch $2k - 1$ and $O(n^{1+1/k})$ edges in expected time $O(|E|)$.

2.2 Implications of Theorem 1

2.2.1 Standard Centralized Model and PRAM

The currently sparsest spanners which can be constructed in linear time are those of Halperin and Zwick [HZ96]. They provide for any $k \geq 1$, a deterministic algorithm running in $O(m)$ time, that produces a spanner with $n^{1+1/k} + n$ edges. We can improve their result for a wide range of k , albeit with a randomized algorithm. First we show a near-linear time algorithm (which can be also executed in parallel), that provides a spanner sparser than Halperin and Zwick in the range $k \geq 2 \ln n / \ln \ln n$.⁷

Corollary 7. *For any unweighted graph $G = (V, E)$ on n vertices and m edges, and any integer $k \geq 2$, there is a randomized algorithm, that with high probability⁸ computes a spanner for G with stretch $2k - 1$ and $n^{1+1/k} \cdot \left(1 + \frac{O(\ln k)}{k}\right)$ edges. The algorithm has $O(k^2 \ln n \ln^* n)$ depth and the running time (or work) is $\tilde{O}(k|E|)$.*

Proof. Apply Theorem 1 with parameters $c = k$ and $\delta = 1/k$. So with probability at least $\frac{k-1}{k} \cdot \frac{1}{k+1} \geq \frac{1}{3k}$ we obtain a spanner whose number of edges is at most

$$(1 + 1/k) \cdot \frac{(kn)^{1+1/k}}{k-1} \leq n^{1+1/k} \cdot \left(1 + \frac{O(\ln k)}{k}\right). \quad (4)$$

Run the algorithm $C \cdot k \ln n$ times for some constant C . We noted in Section 2.1.1 that each run takes $O(k \ln^* n)$ depth and $O(|E|)$ work, so the time bounds are as promised. Now, with probability at least $1 - (1 - 1/(3k))^{C \cdot k \ln n} \geq 1 - n^{-C/3}$, we achieved a spanner with number of edges as in (4) in one of the executions. $\square$

Remark 1. *Whenever $k \geq 2 \ln n / \ln \ln n$, we have $n^{1/k} \leq \sqrt{\ln n}$, so the number of edges in Corollary 7 is $n^{1+1/k} + o(n)$, and the running time is $\tilde{O}(k|E|)$.*

2.2.2 Distributed Model

In a distributed setting we have the following result.

Corollary 8. *For any unweighted graph $G = (V, E)$ on n vertices, any $k \geq 1$ and $0 < \epsilon < 1$, there is a randomized distributed algorithm that with probability at least $1 - \epsilon$ computes a spanner with stretch $2k - 1$ and $O(n/\epsilon)^{1+1/k}$ edges, within k rounds.*

Proof. Apply Theorem 1 with $c = 3/\epsilon$ and $\delta = 2/\epsilon$, so the success probability is at least

$$(1 - \epsilon/3) \cdot (1 - \epsilon/(\epsilon + 2)) > 1 - \epsilon.$$

With these parameters, by Theorem 1, the number of edges in spanner will be bounded by $O(n/\epsilon)^{1+1/k}$. $\square$

⁷In fact, the factor 2 can be replaced by any $1 + \epsilon$ for constant $\epsilon > 0$.

⁸By high probability we mean probability at least $1 - n^{-C}$, for any desired constant C .

2.2.3 Ultra-Sparse Spanners and Skeletons

We now show that in the regime $k \geq \ln n$, our algorithm (that succeeds with probability close to 1) provides a spanner whose number of edges is very close to n (as a function of k and the success probability). This will hold in all computational models we considered. We note that for the centralized and PRAM models, Corollary 7 gives high probability with roughly the same sparsity, albeit with larger depth and work.

Corollary 9. *For any unweighted graph $G = (V, E)$ on n vertices, and any integer $k \geq \ln n$ and parameter $2/k < \epsilon < 1$, there is a randomized algorithm, that with probability at least $1 - \epsilon$ computes a spanner for G with stretch $2k - 1$ and $n \cdot \left(1 + \frac{O(\ln n)}{\epsilon \cdot k}\right)$ edges. The number of rounds in distributed model is k , in PRAM it is $O(k \log^* n)$ depth and $O(|E|)$ work, and in the centralized model it is $O(|E|)$ time.*

Proof. Apply Theorem 1 with parameters $c = k$ and $\delta = 2/\epsilon$, so with probability at least $\frac{k-1}{k} \cdot \frac{2/\epsilon}{2/\epsilon+1} \geq 1 - \epsilon$ we obtain a $(2k - 1)$ -spanner. In the regime $k \geq \ln n$ we have $(kn)^{1/k} \leq e^{(2 \ln n)/k} \leq 1 + O(\ln n)/k$, so the number of edges is at most

$$(1 + 2/\epsilon) \cdot \frac{(kn)^{1+1/k}}{k-1} - 2(n-1)/\epsilon \leq n \cdot \left(1 + \frac{O(\ln n)}{\epsilon \cdot k}\right). \quad (5)$$

□

Remark 2. *The spanner of Corollary 9 can be used as a skeleton. E.g., one can take $\epsilon = o(1)$ and $k = \tilde{O}(\log n)$, to obtain with probability $1 - o(1)$, a skeleton with $n(1 + o(1))$ edges, which is computed in $\tilde{O}(\log n)$ rounds.*

2.3 Weighted Graphs

Miller et al. [MPVX15] used their efficient algorithm for constructing $O(k)$ -spanners with $O(n^{1+1/k})$ edges for unweighted graphs, to provide an efficient algorithm for constructing $O(k)$ -spanners with $O(n^{1+1/k} \log k)$ edges for weighted graphs. In this section we argue that their scheme can be used to convert our algorithm for constructing $(2k - 1)$ -spanners with $O(n^{1+1/k})$ edges for unweighted graphs (Theorem 1; see also Section 2.1.1) into an efficient algorithm for constructing $(2k - 1)(1 + \epsilon)$ -spanners with $O(n^{1+1/k} \cdot \frac{\log k}{\epsilon})$ edges for weighted graphs.

The scheme of [MPVX15] works in the following way. It partitions all edges of $G = (V, E)$ into $\lceil \log_{1+\epsilon} \omega_{\max} \rceil = \lambda$ categories $E_t = \{e \in E \mid \omega(e) \in [(1 + \epsilon)^{t-1}, (1 + \epsilon)^t)\}$, $t = 1, 2, \dots, \lambda$. (We assume that the minimum weight is 1, and the maximum weight is $\omega_{\max}$. The last category E_λ should also contain edges of weight exactly $\omega_{\max}$.) Now one defines $\ell = \lceil \log_{1+\epsilon} k^c \rceil$, for a sufficiently large constant c , graphs $G_j = (V, \hat{E}_j)$, $j = 0, 1, \dots, \ell - 1$, $\hat{E}_j = \bigcup \{E_t \mid t \equiv j \pmod{\ell}\}$.

Observe that the edge weights in (each) G_j are *well-separated*, i.e., $\hat{E}_j$ is a disjoint union of at most $q = \lceil \lambda/\ell \rceil$ edge sets $E^{(1)}, \dots, E^{(q)}$, such that the edge weights within each set are within a factor of $1 + \epsilon$ from one another. Moreover, if edge weights in $E^{(i)}$, $i = 1, 2, \dots, q$, are in the range $[w^{(i)}, (1 + \epsilon)w^{(i)}]$, then we have $w^{(i)} = w^{(1)}(k^c)^{i-1}$, for every $i = 1, 2, \dots, q$.

For each graph G_j , the scheme of [MPVX15] constructs an $O(k)$ -spanner with $O(n^{1+1/k})$ edges. It then takes a union of $\ell = O(\frac{\log k}{\epsilon})$ such spanners as the ultimate $O(k)$ -spanner of the original graphs. (In fact, [MPVX15] used specifically $\epsilon = 1$.) We will next outline the way in which [MPVX15] construct $O(k)$ -spanner with $O(n^{1+1/k})$ edges for each G_j , and show how to modify it to provide a $(2k - 1)(1 + \epsilon)$ -spanner.

The scheme starts with running a routine of [MPVX15] that constructs an $O(k)$ -spanner $H^{(1)}$ with $O(n^{1+1/k})$ edges for the *unweighted* graph $(V^{(1)}, E^{(1)})$, $V^{(1)} = V$, and constructing the exponential start time partition $\mathcal{P}^{(1)}$ for it. It then contracts each of the clusters of $\mathcal{P}^{(1)}$ (which have unweighted radii at most $k - 1$) into single vertices of $V^{(2)}$, and runs the unweighted spanner routine on $(V^{(2)}, E^{(2)})$. As a result, it constructs an $O(k)$ -spanner $H^{(2)}$ and a partition $\mathcal{P}^{(2)}$ of $V^{(2)}$, contracts all clusters of $\mathcal{P}^{(2)}$ to get $V^{(3)}$, etc. The final spanner returned by the scheme is $H = \bigcup_{i=1}^q H^{(i)}$.

The scheme guarantees stretch $(1 + \epsilon)(1 + O(k^{-(c-1)}))O(k)$, because the blackbox routine for unweighted graphs provides stretch $O(k)$ for each category of weights, but the weights are uniform only up to a factor of $1 + \epsilon$. Also, the factor of $1 + O(k^{-(c-1)})$ appears, because one contracts clusters of unweighted diameter $O(k)$ of lower scales, on which all edge weights are a factor of roughly k^{-c} smaller than the edge weights on the current scale.

In the analysis of $|H|$, [MPVX15] show that every vertex u is active (i.e., non-isolated vertex which is not yet contracted into a larger super-vertex) for expected $O(n^{1/k})$ phases, and when it is active, it contributes expected $O(n^{1/k})$ edges to the spanner of the current phase. Hence the overall size of the spanner is $O(n^{1+2/k})$, and by rescaling $k' = 2k$, they ultimately get their result. (See the proof of Theorem 3.3 in [MPVX15] for full details of this proof. We have sketched it for the sake of completeness.)

While the stretch analysis of [MPVX15] is sufficiently precise for our purposes, this is not the case with the size analysis. Indeed, even when one plugs in stretch $2k - 1$ of our unweighted spanner routine instead of stretch $O(k)$ of their routine, still one obtains a $(2k - 1)(1 + \epsilon)^2(1 + k^{-(c-1)})$ -spanner with $O(n^{1+2/k})$ edges, i.e., a $(4k - 2)(1 + O(\epsilon))$ -spanner with $O(n^{1+1/k})$ edges (for each G_j).

In what follows we refine their size analysis, and show that, in fact, every vertex u contributes expected $O(n^{1/k})$ edges in *all phases* of the algorithm *altogether* (for a single graph G_j with well-separated edge weights). Denote by $r_u^{(i)}$ the radius that u tosses from $\mathcal{E}\mathcal{X}\mathcal{P}(\beta)$ in the i th phase, $i = 1, 2, \dots, q$, assuming that it is active on that phase. We say that a vertex v (which is active on phase i) is a *candidate* vertex of phase i if its broadcast message reaches u no later than within one time unit after the time $-r_u^{(i)}$, i.e., $-r_u^{(i)} + 1 \geq -r_v^{(i)} + d(v, u)$, where $d(v, u)$ is the unweighted distance between v and u in the graph on which the unweighted spanner routine is invoked on phase i .

Let $X^{(i)}$ denote the random variable counting the number of such candidate vertices on phase i , for $i = 1, 2, \dots, q$. (Recall that these are the vertices which might cause u to add an edge to the spanner.) Let j denote the random variable which is the phase in which u was contracted (and $j = q$ if there is no such phase). That is, j indicates the level in which the broadcast of some candidate vertex v has $-r_v^{(j)} + d(v, u) < -r_u^{(i)}$. Denote also by $\hat{X}^{(i)}$, $i = 1, 2, \dots, q$, the total number of candidates u sees between the beginning of phase i , and up until phase j , where a candidate vertex v reaches u before time $-r_u^{(j)}$. On that phase j , $\hat{X}^{(i)}$ counts the number of candidates with index not larger than that of the candidate vertex v (assume every vertex has an arbitrary distinct index in $\{1, \dots, n\}$). Note that for $i > j$, by definition, $\hat{X}^{(i)} = 0$. Also, in particular, $\hat{X} = \hat{X}^{(1)}$ is at least the total contribution of edges u adds to the spanner, except for up to expected $O(n^{1/k})$ edges that it might contribute on phase j (as shown in Lemma 2).

We next argue that for any $t = 0, 1, 2, \dots$,

$$\Pr(\hat{X} > t) \leq (1 - e^{-\beta})^t. \quad (6)$$

This implies that

$$\mathbb{E}(\hat{X}) = \sum_{t=0}^{\infty} \Pr(\hat{X} > t) \leq \sum_{t=0}^{\infty} (1 - e^{-\beta})^t = e^{\beta} = O(n^{1/k}).$$

First, note that

$$\Pr(\hat{X} > t) = \sum_{t^{(1)}=0}^{\infty} \Pr(X^{(1)} = t^{(1)}) \cdot \Pr(\hat{X} > t \mid X^{(1)} = t^{(1)}).$$

For $t^{(1)} > t$, we have

$$\Pr(\hat{X} > t \mid X^{(1)} = t^{(1)}) = (1 - e^{-\beta})^t.$$

To justify this equation, note that the left-hand side is exactly the probability that on the first phase, all the t first candidate vertices v have $-r_v^{(1)} + d(v, u) \geq -r_u^{(1)}$, conditioned on them being candidates, i.e., on $-r_v + d(v, u) \leq -r_u^{(1)} + 1$. Since these are independent shifted exponential random variables, the equation follows from the memoryless property of the exponential distribution.

For $t^{(1)} \leq t$, we have

$$\begin{aligned} \Pr(\hat{X} > t \mid X^{(1)} = t^{(1)}) &= (1 - e^{-\beta})^{t^{(1)}} \cdot \Pr(\hat{X}^{(2)} > t - t^{(1)} \mid X^{(1)} = t^{(1)}, \hat{X}^{(1)} \geq t^{(1)}) \\ &= (1 - e^{-\beta})^{t^{(1)}} \cdot \Pr(\hat{X}^{(2)} > t - t^{(1)} \mid \hat{X}^{(1)} \geq X^{(1)}). \end{aligned}$$

(Again, $(1 - e^{-\beta})^{t^{(1)}}$ is the probability that no candidate vertex of the first phase reached u before time $-r_u^{(1)}$, and so u was not contracted away at this phase.)

We conduct an induction on the phase, where the induction base is the last phase $i = q$. On the last phase, for any h ,

$$\Pr(\hat{X}^{(q)} > h \mid \hat{X}^{(1)} \geq X^{(1)}, \hat{X}^{(2)} \geq X^{(2)}, \dots, \hat{X}^{(q-1)} \geq X^{(q-1)}) \leq (1 - e^{-\beta})^h,$$

because it is just the probability that none of the first h candidates (that have $-r_v^{(q)} + d(v, u) \leq -r_u^{(1)} + 1$) reaches u before time $-r_u^{(q)}$. (If there are fewer candidates or u was contracted in a previous phase, then this probability is 0.)

Hence by the inductive hypothesis,

$$\Pr(\hat{X}^{(2)} > t - t^{(1)} \mid \hat{X}^{(1)} \geq X^{(1)}) \leq (1 - e^{-\beta})^{t-t^{(1)}},$$

and so

$$\Pr(\hat{X}^{(1)} > t \mid X^{(1)} = t^{(1)}) \leq (1 - e^{-\beta})^t,$$

for any $t^{(1)} \leq t$ as well. Hence

$$\begin{aligned} \Pr(\hat{X} > t) &= \sum_{t^{(1)}=0}^{\infty} \Pr(X^{(1)} = t^{(1)}) \cdot \Pr(\hat{X} > t \mid X^{(1)} = t^{(1)}) \\ &\leq \sum_{t^{(1)}=0}^{\infty} \Pr(X^{(1)} = t^{(1)}) \cdot (1 - e^{-\beta})^t = (1 - e^{-\beta})^t, \end{aligned}$$

as required.

Hence the expected contribution of every vertex is $O(n^{1/k})$, and the overall spanner size for each G_j is, in expectation, $O(n^{1+1/k})$. The running time of the algorithm is expected to be $O(|E|)$, following the analysis of [MPVX15]. Moreover, as in [MPVX15], our algorithm can be implemented in PRAM model, in $O(\log n \cdot \log^* n \cdot \log \Lambda)$ depth, and $O(|E|)$ work, where Λ is the aspect ratio of the input graph. We summarize the result below.

Theorem 2. *Given a weighted n -vertex graph G , and a pair of parameters $k \geq 1, 0 < \epsilon < 1$, our algorithm computes a $(2k - 1)(1 + \epsilon)$ -spanner of G with $O(n^{1+1/k} \cdot (\log k)/\epsilon)$ edges, in expected $O(|E|)$ centralized time, or in $O(\log n \cdot \log^* n \cdot \log \Lambda)$ depth and $O(|E|)$ work.*

The result of Theorem 2 can be used in conjunction with the scheme of [ES16] to devise an algorithm that computes $(2k - 1)(1 + \epsilon)$ -spanners of size $O(n^{1+1/k}(\log k/\epsilon)1/\epsilon)$, with *lightness* (i.e., weight of the spanner divided by the weight of the MST of the input graph) $O(k \cdot n^{1/k}(1/\epsilon)^{2+1/k})$, in expected time $O(|E| + \min\{n \log n, |E|\alpha(n)\})$, where $\alpha(\cdot)$ is an inverse-Ackermann function. This improves a result of [ES16] that provides spanners with the same stretch and lightness, but with more edges (specifically, $O((k + (1/\epsilon)^{2+1/k})n^{1+1/k})$, and using $O(k \cdot |E| + \min\{n \log n, |E|\alpha(n)\})$ time. Recently, consequently to our work, Alstrup et al. [ADF⁺17] further improved these bounds. We thus omit the details of our argument that provides the aforementioned bounds.

3 An Efficient Centralized Construction of Nearly-Additive Spanners and Emulators

3.1 A Basic Variant of the Algorithm

In this section we present an algorithm for constructing $(1 + \epsilon, \beta)$ -spanners, which can be efficiently implemented in various settings. We start with the centralized setting. In this setting we present two variants of our construction. The first variant presented in this section is somewhat simpler, while the second variant presented in the next section provides better bounds.

Let $G = (V, E)$ be an unweighted graph on n vertices, and let $k \geq 1, \epsilon > 0$ and $0 < \rho < 1/2$ be parameters. Unlike the algorithm of [EP04], our algorithm does not employ sparse partitions of [AP92]. The algorithm initializes the spanner H as an empty set, and proceeds in phases. It starts with setting $\hat{\mathcal{P}}_0 = \{\{v\} \mid v \in V\}$ to be the partition of V into singleton clusters. The partition $\hat{\mathcal{P}}_0$ is the input of phase 0 of our algorithm. More generally, $\hat{\mathcal{P}}_i$ is the input of phase i , for every index i in a certain appropriate range, which we will specify in the sequel.

Throughout the algorithm, all clusters C that we will construct will be centered at designated centers r_C . In particular, each singleton cluster $C = \{v\} \in \hat{\mathcal{P}}_0$ is centered at v . We define $Rad(C) = \max\{d_{G(C)}(r_C, v) \mid v \in C\}$, and $Rad(\hat{\mathcal{P}}_i) = \max_{C \in \hat{\mathcal{P}}_i} \{Rad(C)\}$.

All phases of our algorithm except for the last one consist of two steps. Specifically, these are the *superclustering* and the *interconnection* steps. The last phase contains only the interconnection step, and the superclustering step is skipped. We also partition the phases into two *stages*. The first stage consists of phases $0, 1, \dots, i_0 = \lfloor \log(\kappa\rho) \rfloor$, and the second stage consists of all the other phases $i_0 + 1, \dots, i_1$ where $i_1 = i_0 + \left\lceil \frac{\kappa+1}{\kappa\rho} \right\rceil - 2$, except for the last phase $\ell = i_1 + 1$. The last phase will be referred to as the *concluding phase*.

Each phase i accepts as input two parameters, the distance threshold parameter δ_i , and the degree parameter deg_i . The difference between stage 1 and 2 is that in stage 1 the degree parameter grows exponentially, while in stage 2 it is fixed. The distance threshold parameter grows in the same steady rate (increases by a factor of $1/\epsilon$) all through the algorithm.

Next we describe the first stage of the algorithm. We start with describing its superclustering step. We set $deg_i = n^{2^i/\kappa}$, for all $i = 0, 1, \dots, i_0$. Let $R_0 = 0$, and $\delta_i = (1/\epsilon)^i + 4 \cdot R_i$, where R_i is determined by the following recursion: $R_{i+1} = \delta_i + R_i = (1/\epsilon)^i + 5 \cdot R_i$. We will show that the inequality $Rad(\hat{\mathcal{P}}_i) \leq R_i$ will hold for all i .

On phase i , each cluster $C \in \hat{\mathcal{P}}_i$ is sampled i.a.r. with probability $1/\deg_i$. Let $\mathcal{S}_i$ denote the set of sampled clusters. We now conduct a BFS exploration to depth δ_i in G rooted at the set $S_i = \bigcup_{C \in \mathcal{S}_i} \{r_C\}$. As a result, a forest F_i is constructed, rooted at vertices of S_i . For a cluster center $r' = r_{C'}$ of a cluster $C' \in \hat{\mathcal{P}}_i \setminus \mathcal{S}_i$ such that r' is spanned by F_i , let r_C be the root of the forest tree of F_i to which r' belongs. (The vertex r_C is by itself a cluster center of a cluster $C \in \mathcal{S}_i$.) The cluster C' becomes now superclustered in a cluster $\hat{C}$ centered around the cluster C . (We also say that C' is *associated* with C . We will view association as a transitive relation, i.e., if C' is associated with C and C'' is associated with C' , we will think of C'' as associated with C as well.)

The cluster center r_C of C becomes the new cluster center of $\hat{C}$, i.e., $r_{\hat{C}} = r_C$. The vertex set of the new supercluster $\hat{C}$ is the union of the vertex set of C with the vertex sets of all clusters C' which are superclustered into $\hat{C}$. The edge set $T_{\hat{C}}$ of the new cluster $\hat{C}$ contains the BFS spanning trees of all these clusters, and, in addition, it contains shortest paths from the forest F_i between r_C and each $r_{C'}$ as above. $\hat{\mathcal{S}}_i$ is the set of superclusters created by this process. We set $\hat{\mathcal{P}}_{i+1} = \hat{\mathcal{S}}_i$. All edges that belong to the edgeset of one of these superclusters are now added to the spanner H .

For each supercluster $\hat{C}$, we write $\text{Rad}(\hat{C}) = \text{Rad}(T_{\hat{C}}, r_{\hat{C}})$. Observe that $\text{Rad}(\hat{\mathcal{P}}_0) \leq R_0 = 0$, and $\text{Rad}(\hat{\mathcal{S}}_0) = \max\{\text{Rad}(\hat{C}) \mid \hat{C} \in \hat{\mathcal{S}}_0\} \leq \delta_0 + R_0 = R_1 = 1$. More generally we have

$$\text{Rad}(\hat{\mathcal{S}}_i) = \max\{\text{Rad}(\hat{C}) \mid \hat{C} \in \hat{\mathcal{S}}_i\} \leq \delta_i + \text{Rad}(\hat{\mathcal{P}}_i) \leq \delta_i + R_i \leq (1/\epsilon)^i + 5R_i = R_{i+1}. \quad (7)$$

Denote by $\hat{\mathcal{U}}_i$ the set of clusters of $\hat{\mathcal{P}}_i$ which were not superclustered into clusters of $\hat{\mathcal{S}}_i$. In the interconnection step for $i \geq 1$, every cluster center r_C of a cluster $C \in \hat{\mathcal{U}}_i$ initiates a BFS exploration to depth $\frac{1}{2}\delta_i$, i.e., half the depth of the exploration which took place in the superclustering step. For each cluster center $r_{C'}$ for $C' \in \hat{\mathcal{P}}_i$ which is discovered by the exploration initiated in r_C , the shortest path between r_C and $r_{C'}$ is inserted into the spanner H . The first phase $i = 0$ is slightly different: the exploration depth is set to be 1, and we add an edge from $\{v\} \in \hat{\mathcal{U}}_0$ to all neighbors that are in $\hat{\mathcal{U}}_0$. This completes the description of the interconnection step.

Lemma 10. *For any vertex $v \in V$, the expected number of explorations that visit v at the interconnection step of phase i is at most $\deg_i$.*

Proof. For $i \geq 1$, assume that there are l clusters of $\hat{\mathcal{P}}_i$ whose centers are within distance $\delta_i/2$ from v . If at least one of them is sampled to $\mathcal{S}_i$, then no exploration will visit v (since in the superclustering phase the sampled center will explore to distance δ_i , and thus will supercluster all these centers). The probability that none of them is sampled is $(1 - 1/\deg_i)^l$, in which case we get that l explorations visit v , so the expectation is $l \cdot (1 - 1/\deg_i)^l \leq \deg_i$ (which holds for any l).

For $i = 0$, we note that we add an edge touching v iff none of its neighbors were sampled at phase 0 (as otherwise it would be clustered and thus not in $\hat{\mathcal{U}}_0$). The expected number of edges added is once again $l \cdot (1 - 1/\deg_i)^l \leq \deg_i$ (here l is the number of neighbors). $\square$

We also note the following lemma for future use, its proof follows from a simple Chernoff bound.

Lemma 11. *For any constant $c > 1$, with probability at least $1 - 1/n^{c-1}$, for every vertex $v \in V$, at least one among the $\deg_i \cdot c \cdot \ln n$ closest cluster centers $r_{C'}$ with $C' \in \hat{\mathcal{P}}_i$ to v is sampled, i.e., satisfies $C' \in \mathcal{S}_i$.*

Observe that no vertex $v \in V$ is explored by more than $c \cdot \ln n \cdot \deg_i$ explorations, with probability at least $1 - n^{-(c-1)}$. Indeed, otherwise when $i \geq 1$ there would be more than $c \cdot \ln n \cdot \deg_i$ cluster centers r_C of unsampled clusters $C \in \hat{\mathcal{P}}_i$ at pairwise distance at most δ_i . Applying Lemma 11 to any of them we

conclude that the particular cluster C was superclustered by a nearby sampled cluster, i.e., $C \notin \hat{\mathcal{U}}_i$, contradiction. In the case $i = 0$, we would have that v has at least $c \cdot \ln n \cdot \deg_i$ unsampled neighbors, which occurs with probability at most n^{-c} . Hence, by union-bound, every vertex v is explored by at most $c \cdot \ln n \cdot \deg_i$ explorations, with probability at least $1 - n^{-(c-1)}$.

Lemma 10 suggests that the interconnection step of phase i can be carried out in expected $O(|E| \cdot \deg_i)$ time. Clearly, the superclustering step can be carried out in just $O(|E|)$ time, and thus the running time of the interconnection step dominates the running time of phase i . In order to control the running time, we terminate stage 1 and move on to stage 2 when $i_0 = \lfloor \log(\kappa\rho) \rfloor$, so that $\deg_{i_0} \leq n^\rho$.

Observe also that the superclustering step inserts into the spanner at most $O(n)$ edges (because we insert a subset of edges of F_i , and F_i is a forest), and by Lemma 10 the interconnection step inserts in expectation at most $O(|\hat{\mathcal{P}}_i| \cdot \deg_i \cdot ((1/\epsilon)^i + R_i)) = O(|\hat{\mathcal{P}}_i| \cdot \deg_i \cdot (1/\epsilon)^i)$ edges. (We will soon show that $R_i = O((1/\epsilon)^{i-1})$.) A more detailed argument providing an upper bound on the number of edges inserted by the interconnection step will be given below.

Lemma 12. *For all i , $1 \leq i \leq \ell$, for every pair of clusters $C \in \hat{\mathcal{U}}_i$, $C' \in \hat{\mathcal{P}}_i$ at distance at most $\frac{1}{2}(1/\epsilon)^i$ from one another, a shortest path between the cluster centers of C and C' was inserted into the spanner H . Moreover, for any pair $\{v\} \in \hat{\mathcal{U}}_0$, $\{v'\} \in \hat{\mathcal{P}}_0$, such that $e = (v, v') \in E$, the edge e belongs to H .*

Proof. We start with proving the first assertion of the lemma. For some index i , $1 \leq i \leq \ell$, and a pair $C \in \hat{\mathcal{U}}_i$, $C' \in \hat{\mathcal{P}}_i$ of clusters, let $r_C, r_{C'}$ be the respective cluster centers. Then we have

$$\begin{aligned} d_G(r_C, r_{C'}) &\leq \text{Rad}(C) + d_G(C, C') + \text{Rad}(C') \\ &\leq d_G(C, C') + 2 \cdot R_i \\ &\leq \frac{1}{2}(1/\epsilon)^i + 2 \cdot R_i = \frac{1}{2}\delta_i, \end{aligned}$$

and so a shortest path between r_C and $r_{C'}$ was inserted into the spanner H .

The second assertion of the lemma is guaranteed by the interconnection step of phase 0. $\square$

Next we analyze the radii of clusters' collections $\hat{\mathcal{P}}_i$, for $i = 1, 2, \dots$

Lemma 13. *For $i = 0, 1, \dots, \ell$, the value of R_i is given by*

$$R_i = \sum_{j=0}^{i-1} (1/\epsilon)^j \cdot 5^{i-1-j}.$$

Proof. The proof is by induction of the index i . The basis ($i = 0$) is immediate as $R_0 = 0$. For the induction hypothesis, note that

$$\begin{aligned} R_{i+1} &= \delta_i + R_i = (1/\epsilon)^i + 5 \cdot R_i \\ &= (1/\epsilon)^i + 5 \cdot \left(\sum_{j=0}^{i-1} (1/\epsilon)^j \cdot 5^{i-1-j} \right) \\ &= \sum_{j=0}^i (1/\epsilon)^j \cdot 5^{i-j}, \end{aligned}$$

as required. $\square$

Observe that Lemma 13 implies that for $\epsilon < 1/10$, we have $R_i = 5^{i-1} \cdot \frac{1/(5\epsilon)^{i-1}-1}{1/(5\epsilon)-1} \leq \frac{1}{1-5\epsilon} \cdot (1/\epsilon)^{i-1} \leq 2 \cdot (1/\epsilon)^{i-1}$. Recall that $\hat{\mathcal{P}}_i = \hat{\mathcal{S}}_{i-1}$. Hence $\text{Rad}(\hat{\mathcal{P}}_i) = \text{Rad}(\hat{\mathcal{S}}_{i-1})$. By inequality (7), we have $\text{Rad}(\hat{\mathcal{P}}_i) \leq (1/\epsilon)^{i-1} + 5 \cdot R_{i-1} = R_i$, for all $i = 0, 1, \dots, \ell$.

We analyze the number of clusters in collections $\hat{\mathcal{P}}_i$ in the following lemma.

Lemma 14. For $i = 0, 1, \dots, i_0$,

$$|\hat{\mathcal{P}}_i| \leq 2 \cdot n^{1-\frac{2^i-1}{\kappa}}, \quad (8)$$

with probability at least $1 - \exp\{-\Omega(n^{1-\frac{2^i-1}{\kappa}})\}$.

Proof. The probability that a vertex $v \in V$ will be a center of a cluster in $\hat{\mathcal{P}}_i$ is $\prod_{j=0}^{i-1} 1/\deg_j = n^{-(2^i-1)/\kappa}$. Thus the expected size of $\hat{\mathcal{P}}_i$ is $n^{1-(2^i-1)/\kappa}$, and by Chernoff bound,

$$\Pr[|\hat{\mathcal{P}}_i| \geq 2\mathbb{E}[|\hat{\mathcal{P}}_i|]] \leq \exp\{-\Omega(\mathbb{E}[|\hat{\mathcal{P}}_i|])\} = \exp\{-\Omega(n^{1-\frac{2^i-1}{\kappa}})\}.$$

□

Since for $\rho < 1/2$ and $i \leq i_0 = \lfloor \log(\kappa\rho) \rfloor$, we have $n^{1-\frac{2^{i+1}-1}{\kappa}} \geq n^{1-2\rho} = \omega(\log n)$, we conclude that whp for all $0 \leq i \leq i_0$, $|\hat{\mathcal{P}}_{i+1}| = |\hat{\mathcal{S}}_i| = O(n^{1-\frac{2^{i+1}-1}{\kappa}})$. Hence in particular, $|\hat{\mathcal{P}}_{i_0+1}| = O(n^{1-\rho+1/\kappa})$, whp. The total expected running time of the first stage is at most

$$O(|E|) \sum_{i=1}^{i_0} \deg_i = O(|E| \cdot n^{\frac{2^{i_0}}{\kappa}}) = O(|E| \cdot n^\rho).$$

Since each superclustering step inserts at most $O(n)$ edges into the spanner, the overall number of edges inserted by the i_0 superclustering steps of stage 1 is $O(n \cdot i_0) = O(n \cdot \log(\kappa\rho))$. The expected number of edges added to the spanner by the interconnection step of phase i is at most

$$O(|\hat{\mathcal{P}}_i| \cdot \deg_i \cdot (1/\epsilon)^i) = O(n^{1-\frac{2^i-1}{\kappa}} \cdot (1/\epsilon)^{\log(\kappa\rho)} \cdot n^{\frac{2^i}{\kappa}}) = O(n^{1+1/\kappa} \cdot (1/\epsilon)^{\log(\kappa\rho)}).$$

Next we describe stage 2 of the algorithm, i.e., phases $i = i_0+1, i_0+2, \dots, i_1$, where $i_1 = i_0 + \lceil \frac{\kappa+1}{\kappa\rho} \rceil - 2$. All these phases are executed with the same fixed degree parameter $\deg_i = n^\rho$. On the other hand, the distance threshold keeps growing in the same steady rate as in stage 1, i.e., it is given by $\delta_i = (1/\epsilon)^i + 4R_i$. The sets $\hat{\mathcal{P}}_{i_0+1}, \hat{\mathcal{P}}_{i_0+2}, \dots, \hat{\mathcal{P}}_{i_1}$ on which phases $i_0+1, i_0+2, \dots, i_1$, respectively, operate are defined by $\hat{\mathcal{P}}_{i_0+i} = \hat{\mathcal{S}}_{i_0+i-1}$, for $1 \leq i \leq i_1 - i_0$.

Lemma 13 keeps holding for these additional $i_1 - i_0$ phases, i.e.,

$$\text{Rad}(\hat{\mathcal{P}}_i) \leq R_i \leq 2 \cdot (1/\epsilon)^{i-1}.$$

(We assume all through that $\epsilon < 1/10$.)

Also, for every pair of clusters $C \in \hat{\mathcal{U}}_i$ and $C' \in \hat{\mathcal{P}}_i$ which are at distance at most $\frac{1}{2}(1/\epsilon)^i$ from one another, their centers are interconnected in the spanner by a shortest path between them.

In addition, for every $i \in [i_0, i_1]$, the expected size of $\hat{\mathcal{P}}_{i+1}$ is

$$\mathbb{E}[|\hat{\mathcal{P}}_{i+1}|] = n \cdot \prod_{j=0}^i 1/\deg_j \leq n^{1+1/\kappa-(i+1-i_0)\rho}.$$

By Chernoff bound, for every such i , with probability at least $1 - \exp\{-\Omega(n^\rho)\}$, we have

$$|\hat{\mathcal{P}}_{i+1}| = |\hat{\mathcal{S}}_i| \leq 2 \cdot n^{1+1/\kappa-\rho-(i-i_0)\rho}.$$

Assuming that $n^\rho = \omega(1)$, we conclude that whp

$$|\hat{\mathcal{P}}_{i_1+1}| = |\hat{\mathcal{S}}_{i_1}| \leq O(n^{1+1/\kappa-(i_1+1-i_0)\rho}) = O(n^{1+1/\kappa-(\lceil \frac{\kappa+1}{\kappa\rho} \rceil - 1)\rho}) = O(n^\rho). \quad (9)$$

(For the assumption above to hold we will need to assume that $\rho \geq \frac{\log \log n}{2 \log n}$, say. We will show soon that this assumption is valid in our setting.)

The time required to perform these $\lceil \frac{\kappa+1}{\kappa\rho} \rceil - 2 \leq 1/\rho$ additional phases is expected to be at most $O(|E| \cdot \deg_i \cdot (1/\rho)) = O(|E|n^\rho/\rho)$.

The final collection of clusters $\hat{\mathcal{P}}_{i_1+1}$ is created by setting $\hat{\mathcal{P}}_{i_1+1} = \hat{\mathcal{S}}_{i_1}$.

We will next bound the expected number of edges inserted into the spanner during stage 2 of the algorithm. Each of the forests F_i , $i \in [i_0 + 1, i_1]$, created during the superclustering steps contributes at most $n - 1$ edges. The interconnection step of phase $i + i_0$ contributes in expectation at most $O(|\hat{\mathcal{P}}_{i+i_0}| \cdot \deg_{i+i_0} \cdot (1/\epsilon)^i) \leq O(n^{1+1/\kappa-i\rho} \cdot (1/\epsilon)^{\log(\kappa\rho)+i})$ edges. Assuming that $1/\epsilon < n^\rho/2$, this becomes a geometric progression, so the overall expected number of edges inserted into the spanner on stage 2 is $O(n^{1+1/\kappa} \cdot (1/\epsilon)^{\log(\kappa\rho)})$. (We will show the validity of this assumption in the end of this section.)

Finally, we describe the concluding phase of the algorithm, i.e., phase $\ell = i_1 + 1$. In this phase we skip the superclustering step (as the number of clusters is already sufficiently small), and proceed directly to the interconnection step.

On this step each of the cluster centers r_C for $C \in \hat{\mathcal{P}}_\ell$ conducts a BFS exploration in G to depth $\frac{1}{2}\delta_\ell = \frac{1}{2}(1/\epsilon)^\ell + 2R_\ell$. (Essentially, we define $\hat{\mathcal{U}}_\ell = \hat{\mathcal{P}}_\ell$, and perform the usual interconnection step of the algorithm.) By (9), the number of edges inserted by this step into the spanner is whp only $O(|\hat{\mathcal{P}}_\ell|^2 \cdot (1/\epsilon)^\ell) = O(|\hat{\mathcal{P}}_{i_1+1}|^2 \cdot (1/\epsilon)^{i_1+1}) = O(n^{2\rho} \cdot (1/\epsilon)^{\log(\rho\kappa)+1/\rho})$. Recall that we assume that $\rho < 1/2$. Hence this number of edges is sublinear in n .

Hence the overall expected number of edges in the spanner is $|H| = O(n^{1+1/\kappa} \cdot (1/\epsilon)^{\log(\kappa\rho)})$. Observe also that the running time of the last phase is $O(|E| \cdot n^\rho)$. Hence the overall expected running time of the algorithm is $O(|E|n^\rho/\rho)$. It remains to analyze the stretch of the resulting spanner H .

Let $\hat{\mathcal{U}} = \bigcup_{j=0}^\ell \hat{\mathcal{U}}_j$. Observe that every singleton cluster $\{v\} \in \hat{\mathcal{P}}_0$ is associated with exactly one cluster of $\hat{\mathcal{U}}$, i.e., $\hat{\mathcal{U}}$ is a partition of V . Note that $\text{Rad}(\hat{\mathcal{U}}_0) = 0$, $\text{Rad}(\hat{\mathcal{U}}_1) \leq 1 = R_1$, and for every $j \in [\ell]$, we have $\text{Rad}(\hat{\mathcal{U}}_j) \leq \text{Rad}(\hat{\mathcal{P}}_j) \leq R_j \leq 2 \cdot (1/\epsilon)^{j-1}$. Denote $c = 2$. Recall also (see Lemma 12) that for $j \geq 1$, for every pair of clusters $C, C' \in \hat{\mathcal{U}}_j$ at distance at most $\frac{1}{2}(1/\epsilon)^j$ from one another, a shortest path between the cluster centers of this pair of clusters in G was added to the spanner H . Moreover, neighboring clusters of $\hat{\mathcal{U}}_0$ are also interconnected by a spanner edge.

Lemma 15. *Consider a pair of indices $0 \leq j < i \leq \ell$, and a pair of neighboring (in G) clusters $C' \in \hat{\mathcal{U}}_j$, $C \in \hat{\mathcal{U}}_i$, and a vertex $w' \in C'$ and the center r of C . Then the spanner H contains a path of length at most $3\text{Rad}(\hat{\mathcal{U}}_j) + 1 + \text{Rad}(\hat{\mathcal{U}}_i)$ between w' and r .*

Proof. Let $(z', z) \in E \cap (C' \times C)$ be an edge connecting this pair of clusters. There exists a subcluster $C'' \subseteq C$, $C'' \in \hat{\mathcal{P}}_j$ such that $z \in C''$. Hence the interconnection step of phase j inserted a shortest path $\pi(r', r'')$ in G between the cluster centers r' of C' and r'' of C'' into the spanner H . Note that the distance between r', r'' is at most $1 + 2\text{Rad}(\hat{\mathcal{P}}_j) \leq 1 + 4(1/\epsilon)^{j-1} < 1/2 \cdot (1/\epsilon)^j$, since we assume $\epsilon < 1/10$. Hence a path between w' and r in H can be built by concatenating a path $\pi(w', r')$ between w' and r' in

the spanning tree $T(C')$ of C' with the path $\pi(r', r'')$ in H , and with the path $\pi(r'', r)$ in the spanning tree $T(C)$ of C . (Note that both r'' and r belong to C .) Its length is at most

$$\begin{aligned} |\pi(w', r')| + |\pi(r', r'')| + |\pi(r'', r)| &\leq \text{Rad}(C') + (\text{Rad}(C') + 1 + \text{Rad}(C'')) + \text{Rad}(C) \\ &\leq 3\text{Rad}(\hat{\mathcal{U}}_j) + 1 + \text{Rad}(\hat{\mathcal{U}}_i). \end{aligned}$$

□

Now we are ready to analyze the stretch of our spanner.

Lemma 16. *Suppose $\epsilon \leq 1/10$. Consider a pair of vertices $u, v \in V$. Fix a shortest path $\pi(u, v)$ between them in G , and suppose that for some index $i \in [0, \ell]$, all vertices of $\pi(u, v)$ are clustered in the set $\hat{\mathcal{U}}^{(i)}$ defined by $\hat{\mathcal{U}}^{(i)} = \bigcup_{j=0}^i \hat{\mathcal{U}}_j$. Then*

$$d_H(u, v) \leq (1 + 16c \cdot \epsilon \cdot i) d_G(u, v) + 4 \sum_{j=1}^i R_j \cdot 2^{i-j}.$$

Proof. The proof is by induction on i . For the induction basis $i = 0$, observe that all vertices of $\pi(u, v)$ are clustered in $\hat{\mathcal{U}}_0$, and thus all edges of $\pi(u, v)$ are inserted into the spanner on phase 0. Hence $d_H(u, v) = d_G(u, v)$.

For the induction step, consider first a pair of vertices x, y such that $|\pi(x, y)| \leq \frac{1}{2}(1/\epsilon)^i$, and $V(\pi(x, y)) \subseteq \hat{\mathcal{U}}^{(i)}$. Let z_1 and z_2 be the leftmost and the rightmost $\hat{\mathcal{U}}_i$ -clustered vertices in $\pi(x, y)$, if exist. (The case when both these vertices exist is the one where the largest stretch is incurred; cf. [EP04].) Let $C_1, C_2 \in \hat{\mathcal{U}}_i$ be their respective clusters, i.e., $z_1 \in C_1, z_2 \in C_2$. Let w_1 (respectively, w_2) be the neighbor of z_1 (resp., z_2) on the subpath $\pi(x, z_1)$ (resp., $\pi(z_2, y)$) of $\pi(x, y)$, and denote by C'_1 and C'_2 the respective clusters of w_1 and w_2 . Observe that $C'_1, C'_2 \in \hat{\mathcal{U}}^{(i-1)}$.

Denote r_1 and r_2 the cluster centers of C_1 and C_2 , respectively. The spanner H contains a path of length at most $d_G(r_1, r_2)$ between these cluster centers. Also, by Lemma 15, since C'_1 and C_1 are neighboring clusters, the spanner H contains a path of length at most $3R_j + 1 + R_i \leq 2R_i + 1$ between w_1 and r_1 , and a path of at most this length between r_2 and w_2 . (For $\epsilon < 1/10$, $3R_j \leq R_i$, for all $j < i$.) Observe also that the subpaths $\pi(x, w_1)$ and $\pi(w_2, y)$ of $\pi(x, y)$ have all their vertices clustered in $\hat{\mathcal{U}}^{(i-1)}$, and thus the induction hypothesis is applicable to these subpaths.

Hence

$$\begin{aligned} d_H(x, y) &\leq d_H(x, w_1) + d_H(w_1, r_1) + d_H(r_1, r_2) + d_H(r_2, w_2) + d_H(w_2, y) \\ &\leq (1 + 16c \cdot \epsilon(i-1)) d_G(x, w_1) + 4 \sum_{j=1}^{i-1} R_j \cdot 2^{i-1-j} + 2R_i + 1 + (d_G(C_1, C_2) + 2R_i) \\ &\quad + 2R_i + 1 + (1 + 16c \cdot \epsilon(i-1)) \cdot d_G(w_2, y) + 4 \sum_{j=1}^{i-1} R_j \cdot 2^{i-1-j} \\ &= (1 + 16c \cdot \epsilon(i-1)) \cdot (d_G(x, w_1) + d_G(w_2, y)) + d_G(C_1, C_2) + 4R_i + 2 + 8 \sum_{j=1}^{i-1} R_j \cdot 2^{i-1-j}. \end{aligned}$$

Note also that

$$d_G(x, y) = d_G(x, w_1) + 1 + d_G(z_1, z_2) + 1 + d_G(w_2, y) \geq d_G(x, w_1) + d_G(C_1, C_2) + 2 + d_G(w_2, y).$$

Hence

$$\begin{aligned}
d_H(x, y) &\leq (1 + 16c \cdot \epsilon(i-1))d_G(x, y) + 4R_i + 8 \sum_{j=1}^{i-1} R_j \cdot 2^{i-1-j} \\
&= (1 + 16c \cdot \epsilon(i-1))d_G(x, y) + 4 \sum_{j=1}^i R_j \cdot 2^{i-j}.
\end{aligned}$$

Now consider a pair of vertices u, v such that all vertices of $\pi(u, v)$ are clustered in $\hat{\mathcal{U}}^{(i)}$, without any restriction on $|\pi(u, v)|$. We partition $\pi(u, v)$ into segments $\pi(x, y)$ of length exactly $\lfloor \frac{1}{2}(1/\epsilon)^i \rfloor$, except maybe one segment of possibly smaller length. Inequality (10) applies to all these segments. Hence

$$\begin{aligned}
d_H(u, v) &\leq (1 + 16c \cdot \epsilon(i-1))d_G(u, v) + 4 \sum_{j=1}^i R_j \cdot 2^{i-j} \lfloor \frac{d_G(u, v)}{\frac{1}{2}(1/\epsilon)^i - 1} \rfloor + 4 \sum_{j=1}^i R_j \cdot 2^{i-j} \\
&\leq \left(1 + 16c \cdot \epsilon(i-1) + \frac{4 \sum_{j=1}^i R_j \cdot 2^{i-j}}{\frac{1}{2}(1/\epsilon)^i - 1} \right) d_G(u, v) + 4 \sum_{j=1}^i R_j \cdot 2^{i-j}.
\end{aligned}$$

It remains to argue that $\frac{8 \sum_{j=1}^i R_j 2^{i-j}}{(1/\epsilon)^i - 2} \leq 16c \cdot \epsilon$. Recall that for every j , we have $R_j \leq c \cdot (1/\epsilon)^{j-1}$. Since $1/\epsilon \geq 10$, the left-hand-side is at most

$$10c \cdot \epsilon^i \sum_{j=1}^i (1/\epsilon)^{j-1} \cdot 2^{i-j} = 10c \cdot \epsilon \sum_{j=1}^i (2\epsilon)^{i-j} = 10c \cdot \epsilon \sum_{h=0}^{i-1} (2\epsilon)^h \leq 16c \cdot \epsilon.$$

□

Observe that (as $\epsilon \leq 1/10$), we have

$$\begin{aligned}
\sum_{j=1}^i R_j \cdot 2^{i-j} &\leq c \sum_{j=1}^i (1/\epsilon)^{j-1} \cdot 2^{i-j} = c \cdot 2^{i-1} \cdot \sum_{j=1}^i \left(\frac{1}{2\epsilon} \right)^{j-1} \\
&= c \cdot 2^{i-1} \sum_{j=0}^{i-1} \left(\frac{1}{2\epsilon} \right)^j = c \cdot 2^{i-1} \frac{(1/2\epsilon)^i - 1}{(1/2\epsilon) - 1} \\
&= c \cdot \epsilon \cdot \frac{\frac{1}{2} \cdot (1/\epsilon)^i - 2^{i-1}}{\frac{1}{2} - \epsilon} = O \left(\left(\frac{1}{\epsilon} \right)^{i-1} \right).
\end{aligned}$$

Note also that the condition of the last lemma holds with $i = \ell$ for every pair $u, v \in V$ of vertices. Hence

Corollary 17. *For every pair $u, v \in V$,*

$$d_H(u, v) \leq (1 + 16c \cdot \ell \cdot \epsilon) d_G(u, v) + O((1/\epsilon)^{\ell-1}).$$

Recall that the spanner H contains, whp, $|H| = O(n^{1+1/\kappa} \cdot \log n \cdot (1/\epsilon)^{\log(\kappa\rho)} + n \cdot \log n \cdot (1/\epsilon)^{\log(\kappa\rho)+1/\rho})$ edges, and the expected running time required to construct it is $O(|E| \cdot n^\rho / \rho)$. Recall also that $\ell = i_1 + 1 \leq \log(\kappa\rho) + 1/\rho + 1$. Set now $\epsilon' = 16c \cdot \ell \cdot \epsilon$. We obtain stretch $\left(1 + \epsilon', O \left(\frac{\log \kappa + 1/\rho}{\epsilon'} \right)^{\log \kappa + 1/\rho} \right)$. The condition $\epsilon < 1/10$ translates now to $\epsilon' \leq 1.6c(\log(\kappa\rho) + 1/\rho)$. We will replace it by a simpler stronger condition $\epsilon \leq 1$.

Corollary 18. For any parameters $0 < \epsilon \leq 1$, $\kappa \geq 2$, and $\rho > 0$, and any n -vertex unweighted graph $G = (V, E)$, our algorithm computes a $(1+\epsilon, \beta)$ -spanner with expected number of edges $O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log \kappa} \cdot n^{1+1/\kappa}$, in expected time $O(|E| \cdot n^\rho/\rho)$, where

$$\beta = \left(\frac{O(\log \kappa + 1/\rho)}{\epsilon}\right)^{\log \kappa + 1/\rho}.$$

A particularly useful setting of parameters is $\rho = 1/\log \kappa$. Then we get a spanner with expected $O\left(\frac{\log \kappa}{\epsilon}\right)^{\log \kappa} \cdot n^{1+1/\kappa}$ edges, in time $O(|E| \cdot n^{\frac{1}{\log \kappa}} \cdot \log \kappa)$, and $\beta = O\left(\frac{\log \kappa}{\epsilon}\right)^{2 \log \kappa}$.

We remark that it makes no sense to set $\rho < 1/\kappa$, as the resulting parameters will be strictly worse than when $\rho = 1/\kappa$. Also our assumptions that $\rho > \log \log n / (2 \log n)$ and $16c \cdot \ell/\epsilon < n^\rho/2$ are justified, as otherwise we get $\beta \geq n$, so a trivial spanner will do.

3.2 An Improved Variant of the Algorithm

In this section we show that the leading coefficient $O((\log \kappa + 1/\rho)/\epsilon)^{\log \kappa}$ of $n^{1+1/\kappa}$ in the size of the spanner can be almost completely eliminated at essentially no price. We also devise here yet sparser constructions of emulators.

For $i = 0, 1, \dots, \ell$, denote by $N_i = |\hat{\mathcal{P}}_i|$ the expected number of clusters which take part in phase i . Recall also that the interconnection step of the i th phase contributes $N_i \cdot \deg_i \cdot \left(\frac{c'\ell}{\epsilon}\right)^i$ edges in expectation, where ℓ is the total number of steps, and c' is a universal constant. Note that the contribution of the interconnection step dominates the contribution of the superclustering step in the current variant of the algorithm, and it will still be the case after the modification that we will now introduce. Hence we will now focus on decreasing the number of edges contributed by the interconnection steps.

We keep the structure of the algorithm intact, and have the values of distance thresholds δ_i unchanged. The only change is in the degree sequence $\deg_0, \deg_1, \dots$ of degree parameters used in phases $0, 1, \dots$, respectively. Next, we describe our new setting of these parameters for stage 1 of the algorithm (i.e., phases i , $1 \leq i \leq i_0$). In the case that $\kappa \geq 16$ let $a = \log \log \kappa$, otherwise, when $\kappa < 16$, let $a = 2$. Define $i_0 = \min\{\lfloor \log(a\kappa\rho) \rfloor, \lfloor \kappa\rho \rfloor\}$, and for $i = 0, 1, \dots, i_0$ let $\deg_i = n^{(2^i-1)/(a\kappa)+1/\kappa}$. We now have that for $i \leq i_0 + 1$,

$$N_i = n \prod_{j=0}^{i-1} 1/\deg_j = n^{1-\frac{2^i-1-i}{a\kappa}-\frac{i}{\kappa}},$$

and in particular, when $i_0 = \lfloor \log(a\kappa\rho) \rfloor$ we have $N_{i_0+1} \leq n^{1-\frac{a\kappa\rho-1-(i_0+1)}{a\kappa}-\frac{i_0+1}{\kappa}} \leq n^{1-\rho}$ (since $a \geq 2$ and $i_0 \geq 1$). Whenever $i_0 = \lfloor \kappa\rho \rfloor$ we also have $N_{i_0+1} \leq n^{1-\frac{i_0+1}{\kappa}} \leq n^{1-\rho}$. Additionally, we always have

$$N_i \cdot \deg_i = n^{1-\frac{2^i-1-i}{a\kappa}-\frac{i}{\kappa}} \cdot n^{\frac{2^i-1}{a\kappa}+\frac{1}{\kappa}} = n^{1+\frac{i}{a\kappa}-\frac{i-1}{\kappa}}.$$

We restrict ourselves to the case that

$$\frac{c'\ell}{\epsilon} \leq n^{\frac{1}{2\kappa}}/2, \quad (10)$$

which holds whenever $\kappa \leq \frac{c_0 \cdot \log n}{\log(\ell/\epsilon)}$, for a sufficiently small constant c_0 . Now the expected number of edges inserted at phase $i \leq i_0$ is at most

$$N_i \cdot \deg_i \cdot \left(\frac{c'\ell}{\epsilon}\right)^i \leq n^{1+\frac{i}{2\kappa}-\frac{i-1}{\kappa}} \cdot \left(\frac{n^{1/(2\kappa)}}{2}\right)^i = n^{1+\frac{1}{\kappa}}/2^i. \quad (11)$$

Thus the total expected number of edges inserted in the first stage is $O(n^{1+1/\kappa})$. The second stage proceeds by setting $\deg_{i_0+1} = n^{\rho/2}$, and in all subsequent phases $i_0 + i$, with $i = 2, 3, \dots, i_1 - i_0$, we have $\deg_i = n^\rho$ as before. The "price" for reducing the degree in the first phase of stage two is that the number of phases i_1 may increase by an additive 1. It follows that $N_{i_0+1} \cdot \deg_{i_0+1} \leq n^{1-\rho/2}$. For $i \geq 2$, at phase $i_0 + i$ we have $N_{i_0+i} \leq n^{1-3\rho/2-(i-2)\rho}$. We set $i_1 = \lfloor 1/\rho \rfloor$, so that $N_{i_1+1} \leq n^{\rho/2}$, and whp we have that $N_{i_1+1} \leq 2n^{\rho/2}$. We calculate

$$N_{i_0+i} \cdot \deg_{i_0+i} \leq n^{1-\rho/2-(i-2)\rho}.$$

Note that $i_0/(2\kappa) \leq \rho/2$, which holds since $i_0 \leq \lfloor \kappa\rho \rfloor$. Hence the condition (10) implies that $(c'\ell/\epsilon)^{i_0} \leq n^{\frac{i_0}{2\kappa}} \leq n^{\rho/2}$. The total expected number of edges inserted at phase $i_0 + 1$ is at most

$$N_{i_0+1} \cdot \deg_{i_0+1} \cdot \left(\frac{c'\ell}{\epsilon}\right)^{i_0+1} \leq n^{1-\rho/2} \cdot n^{\rho/2} \cdot \frac{c'\ell}{\epsilon} \leq n^{1+1/\kappa}.$$

The expected contribution of phase $i_0 + i$ for $i \geq 2$ is at most

$$N_{i_0+i} \cdot \deg_{i_0+i} \cdot \left(\frac{c'\ell}{\epsilon}\right)^{i_0+i} \leq n^{1-\rho/2-(i-2)\rho} \cdot n^{\rho/2} \cdot n^{i/(2\kappa)} / 2^i \leq n^{1+1/\kappa} / 2^i, \quad (12)$$

where the last inequality uses that $\rho \geq 1/\kappa$ (which we may assume w.l.o.g). This implies that the expected number of edges in all these $\lfloor 1/\rho \rfloor$ phases is $O(n^{1+1/\kappa})$.

The upper bound on κ under which this analysis was carried out is $\frac{c_0 \cdot \log n}{\log(\ell/\epsilon)} \geq \frac{\Omega(\log n)}{\log(1/\epsilon) + \log(1/\rho) + \log^{(3)} n}$.⁹ We summarize this discussion with the following theorem.

Theorem 3. *For any unweighted graph $G = (V, E)$ with n vertices, $0 < \epsilon < 1/10$, $2 \leq \kappa \leq \frac{c \cdot \log n}{\log(1/\epsilon) + \log(1/\rho) + \log^{(3)} n}$ for a constant c , and $1/\kappa \leq \rho < 1/2$, our algorithm computes a $(1 + \epsilon, \beta)$ -spanner with $\beta = O(\frac{1}{\epsilon}(\log \kappa + 1/\rho))^{\log \kappa + 1/\rho + \max\{1, \log^{(3)} \kappa\}}$ and expected number of edges $O(n^{1+1/\kappa})$. The expected running time is $O(|E| \cdot n^\rho / \rho)$.*

Note that the sparsest this spanner can be is $O(n \log \log n)$, and at this level of sparsity its $\beta = O(\log \log n + 1/\rho)^{\log \log n + 1/\rho}$. (To get this bound we set $\epsilon > 0$ to be an arbitrary small constant, and $\kappa = \frac{c_0 \log n}{\log^{(3)} n}$.)

This is sparser than the state-of-the-art efficiently-computable sparsest $(1 + \epsilon, \beta)$ -spanner due to [Pet10], which has $O(n(\log \log n)^\phi)$ edges, where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio. Moreover, this spanner has a smaller β than the one of [Pet10] in its sparsest level. Denoting the latter as β_{Pet} , it holds that $\beta_{Pet} \approx O(\log \kappa)^{1.44 \log \kappa + 1/\rho}$, i.e., for every setting of the time parameter ρ , the exponent of our β is smaller than that of β_{Pet} .

3.2.1 Sparse Emulator

Finally, we note that if one allows an *emulator* instead of spanner, then we can decrease the size all the way to $O(n)$ when $\kappa = \log n$. To achieve this, we insert single "virtual" edges instead of every path (of length $(c'\ell/\epsilon)^i$) between every pair of cluster centers that we choose to interconnect on phase i , for every i . Analogously, in the superclustering step we also form a supercluster around a center r_C of a cluster C by adding virtual edges $(r_C, r_{C'})$ for each cluster C' associated with C . The weight of each such edge is

⁹We denote $\log^{(k)} n$ as the iterated logarithm function, e.g. $\log^{(3)} n = \log \log \log n$.

defined by $\omega(r_C, r_{C'}) = d_G(r_C, r_{C'})$. The condition (10) was required to obtain converging sequences at (11) and (12), but without the $(c'\ell/\epsilon)^i$ terms, the number of edges already forms a converging sequence at each stage.

Moreover, one can also use for emulators a shorter degree sequence than the one we used for spanners, and as a result to save the additive term of $\log^{(3)} \kappa$ in the exponent of β . Specifically, one can set $\deg_i = n^{\frac{2^i}{\kappa}}/2^{2^i-1}$, for each $i = 0, 1, \dots, i_0 = \lfloor \log(\kappa\rho) \rfloor$. As a result we get $N_i = n \cdot \prod_{j=0}^{i-1} 1/\deg_j = n^{1-\frac{2^i-1}{\kappa}}$. 2^{2^i-1-i} , and thus the expected number of edges inserted at phase $i \leq i_0$ is at most

$$N_i \cdot \deg_i = n^{1+1/\kappa}/2^i.$$

As before, when the first stage concludes, we run one phase with $\deg_{i_0+1} = n^{\rho/2}$, and all subsequent phases with $\deg_i = n^\rho$. To bound the expected number of edges added at phase $i_0 + 1$ we need to note that $2^{2^{i_0+1}} \leq 2^{2\kappa\rho} \leq n^{\rho/2}$ as long as $\kappa \leq (\log n)/4$. (The latter can be assumed without affecting any of the parameters by more than a constant factor). It follows that $N_{i_0+1} \cdot \deg_{i_0+1} = n^{1-\frac{2^{i_0+1}-1}{\kappa}} \cdot 2^{2^{i_0+1}-1-(i_0+1)} \cdot n^{\rho/2} \leq n^{1+1/\kappa}$. In the remaining phases $N_{i_0+i} \leq n^{1+1/\kappa-(i-1)\rho}$ for $i \geq 2$, and the contribution of these phases is a converging sequence. We conclude the discussion with the following theorem.

Theorem 4. *For any unweighted graph $G = (V, E)$ with n vertices, and for any parameters $0 < \epsilon \leq 1$, $2 \leq \kappa \leq (\log n)/4$, $1/\kappa \leq \rho < 1/2$, our algorithm computes a $(1 + \epsilon, \beta)$ -emulator with $\beta = O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log \kappa + 1/\rho}$ and expected number of edges $O(n^{1+1/\kappa})$. The expected running time is $O(|E| \cdot n^\rho/\rho)$.*

In particular, the algorithm produces a *linear-size* $(1+\epsilon, \beta)$ -emulator with $\beta = O\left(\frac{\log \log n + 1/\rho}{\epsilon}\right)^{\log \log n + 1/\rho}$ within this running time.

3.3 Distributed and Streaming Implementations

In this section we provide efficient distributed and streaming algorithms for constructing sparse $(1 + \epsilon, \beta)$ -spanners. The distributed algorithm works in the CONGEST model.

To implement phase 0, each vertex selects itself into S_0 with probability $n^{-1/\kappa}$, i.a.r.. In distributed model vertices of S_0 send messages to their neighbors. Each vertex u that receives at least one message, picks an origin $v \in S_0$ of one of these messages, and joins the cluster centered at v . It also sends negative acknowledgements to all its other neighbors from S_0 . All unclustered vertices z insert all edges incident on them into the spanner.

It is also straightforward to implement this in $O(1)$ passes in the streaming model.

Each consecutive phase is now also implemented in a straightforward manner, i.e., BFS explorations to depth δ_i in the superclustering steps are implemented via broadcasts and convergecasts in the distributed model, and by δ_i passes in the streaming model. In the interconnection steps, however, we need to implement many BFS explorations which may explore the same vertices. However, by Lemma 11 every vertex is explored on phase i by up to $O(\deg_i \cdot \log n)$ explorations whp, it follows that in distributed setting this step requires, whp, $O(\deg_i \cdot \log n \cdot \delta_i)$ time. Also note that $\sum_i \delta_i = O(\beta)$.

In the streaming model we have two possible tradeoffs. The first uses expected $O(n \cdot \deg_i)$ space to maintain for each vertex v the BFS parents and distance estimates, and requires just δ_i passes. To see that such space suffices, recall that the expected number of explorations which visit any vertex v is at most $\deg_i$, by Lemma 10. Whp, the space is $O(n \cdot \deg_i \cdot \log n) = O(n^{1+\rho} \cdot \log n)$.

The second option in the streaming algorithm is to divide the interconnection step of phase i to $c \cdot \deg_i \cdot \log n$ subphases, for a sufficiently large constant c . On each subphase each exploration source, which was not sampled on previous subphases, samples itself i.a.r. with probability $1/\deg_i$. Then the sampled exploration sources conduct BFS explorations to depth $\delta_i/2$. For every vertex v , the expected number of explorations that traverse it on each subphase is $O(\log n)$. Moreover, by Chernoff's inequality, whp, no vertex v is ever traversed by more than $O(\log n)$ explorations. (Here we take a union-bound on all vertices, all phases, and all subphases. The bad events are that some vertex is traversed by more than twice its expectation explorations on some subphase.) Hence each subphase requires $O(\delta_i)$ passes, and whp, the space requirement is $O(n \log n)$, plus the size of the spanner. After $c \cdot \deg_i \cdot \log n$ subphases, whp, each exploration source is sampled on at least one of the subphases, and so the algorithm performs all the required explorations.

Finally, the stretch analysis of distributed and streaming variants of our algorithm remains the same as in the centralized case.

Hence we obtain the following distributed and streaming analogues of Theorem 3.

Theorem 5. *For any unweighted graph $G = (V, E)$ with n vertices, $0 < \epsilon < 1/10$, $2 \leq \kappa \leq \frac{c \log n}{\log(1/\epsilon) + \log(1/\rho) + \log^{(3)} n}$ for a constant c , and $1/\kappa \leq \rho < 1/2$, our distributed algorithm (CONGEST model) computes a $(1 + \epsilon, \beta)$ -spanner with $\beta = O(\frac{1}{\epsilon}(\log \kappa + 1/\rho))^{\log \kappa + 1/\rho + \max\{1, \log^{(3)} \kappa\}}$ and expected number of edges $O(n^{1+1/\kappa})$. The required number of rounds is whp $O(n^\rho/\rho \cdot \beta \cdot \log n)$.*

Our streaming algorithm computes a spanner with the above properties, in either: $O(n \log n + n^{1+1/\kappa})$ expected space and $O(n^\rho/\rho \cdot \log n \cdot \beta)$ passes, or using $O(n^{1+\rho} \cdot \log n)$ space, whp, and $O(\beta)$ passes.

The streaming algorithm described above can also be modified to provide a $(1 + \epsilon, \beta)$ -emulator as in Theorem 4, within the same pass and space complexities.

4 Applications

In this section we describe applications of our improved constructions of spanners and emulators to computing approximate shortest paths for a set $S \times V$ of vertex pairs, for a subset $S \subseteq V$ of designated sources.

4.1 Centralized Setting

We start with the centralized setting. Here our input graph $G = (V, E)$ is unweighted, and we construct a $(1 + \epsilon, \beta)$ -emulator H of G with $O(n^{1+1/\kappa})$ edges, in expected time $O(|E| \cdot n^\rho/\rho)$, where

$$\beta = O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log \kappa + 1/\rho}. \quad (13)$$

Observe that all edge weights in H are integers in the range $[1, \beta]$. We round all edge weights up to the closest power of $1 + \epsilon$. Let H' be the resulting emulator. Note that for any pair u, v of vertices, we have

$$\begin{aligned} d_G(u, v) &\leq d_H(u, v) \leq d_{H'}(u, v) \leq (1 + \epsilon)d_H(u, v) \\ &\leq (1 + \epsilon)^2 d_G(u, v) + (1 + \epsilon)\beta. \end{aligned}$$

For a sufficiently small ϵ , $(1 + \epsilon)^2 \leq 3\epsilon$, and we rescale $\epsilon' = 3\epsilon$. As a result the constant factor hidden by the O -notation in the basis of β 's exponent grows, but other than that H' has all the properties of the emulator

H . Also, it employs only

$$t = O\left(\frac{\log \beta}{\epsilon}\right) = O\left(\frac{(\log 1/\epsilon + \log(\log \kappa + 1/\rho)) \cdot (\log \kappa + 1/\rho)}{\epsilon}\right)$$

different edge weights. Hence a single-source shortest path computation in H can be performed in $O(|H| + n \log t) = O(n^{1+1/\kappa} + n(\log(1/\epsilon) + \log(\log \kappa + 1/\rho)))$ time [OMSW10]. (See also [KMP11], Section 5.) Hence computing $S \times V$ $(1 + \epsilon, \beta)$ -approximate shortest distances requires $O(|E| \cdot n^\rho/\rho + |S|(n^{1+1/\kappa} + n(\log(1/\epsilon) + \log(\log \kappa + 1/\rho))))$ time.

Theorem 6. *For any n , and for any parameters $0 < \epsilon \leq 1$, $2 \leq \kappa \leq (\log n)/4$, $1/\kappa \leq \rho \leq 1/2$, and any n -vertex unweighted graph $G = (V, E)$ with a set $S \subseteq V$, our algorithm computes $(1 + \epsilon, \beta)$ -approximate $S \times V$ shortest distances in the centralized model in expected $O(|E| \cdot n^\rho/\rho + |S|(n^{1+1/\kappa} + n(\log(1/\epsilon) + \log(\log \kappa + 1/\rho))))$ time, where β is given by (13).*

If one is interested in actual paths rather than just in distances, then one can use our $(1 + \epsilon, \beta)$ -spanner with

$$\beta = O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log \kappa + 1/\rho + \max\{1, \log^{(3)} \kappa\}}, \quad (14)$$

and $O(n^{1+1/\kappa})$ edges, but restricting $\kappa \leq \frac{O(\log n)}{\log(1/\epsilon) + \log(1/\rho) + \log^{(3)} n}$. After computing the spanner H with these properties, we conduct BFS explorations on H originated at each vertex of S . The overall running time becomes $O(|E| \cdot n^\rho/\rho + |S| \cdot n^{1+1/\kappa})$.

Corollary 19. *For any n , and for any parameters $0 < \epsilon \leq 1$, $\kappa \leq \frac{O(\log n)}{\log(1/\epsilon) + \log(1/\rho) + \log^{(3)} n}$, $1/\kappa \leq \rho \leq 1/2$, and any n -vertex unweighted graph $G = (V, E)$, our algorithm computes $(1 + \epsilon, \beta)$ -approximate $S \times V$ shortest paths in the centralized model in expected $O(|E| \cdot n^\rho/\rho + |S| \cdot n^{1+1/\kappa})$ time, with β given by (14).*

A useful setting of parameters is $\rho = \frac{1}{\log \kappa}$. Then the running time of our algorithms from Theorem 6 and Corollary 19 become respectively $O(|E| \cdot n^{1/\log \kappa} \cdot \log \kappa + |S|(n^{1+1/\kappa} + n(\log 1/\epsilon + \log \log \kappa)))$ and $O(|E| \cdot n^{1/\log \kappa} \cdot \log \kappa + |S| \cdot n^{1+1/\kappa})$ (we note that the former has smaller β given by (13), while the latter has slightly larger β given by (14)).

The algorithm of Corollary 19 always outperforms the algorithm which can be derived by using the spanner of [Pet10] within the same scheme. Specifically, the running time of that algorithm is at least $\tilde{O}(|E|n^\rho) + O(|S| \cdot (n^{1+1/\kappa} + n(\frac{\log \kappa}{\epsilon})^\phi))$, and the additive error β_{Pet} there is given by

$$\beta_{Pet} = O\left(\frac{\log \kappa + 1/\rho}{\epsilon}\right)^{\log_\phi \kappa + 1/\rho},$$

where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio. So β_{Pet} is typically polynomially larger than the additive error β in our algorithm, e.g., for $\rho = 1/\log \kappa$ we have $\beta_{Pet} \approx \beta^{1.22}$. (Setting ρ to be smaller than $1/\log \kappa$ makes less sense, because then the additive errors β and β_{Pet} deteriorate. At any rate, as ρ tends to 0, the two estimates approach each other.) Also, in the bound of [Pet10] there is a term of $|S| \cdot n \cdot (\frac{\log \kappa}{\epsilon})^\phi$, which does not occur in our construction.

4.2 Streaming Setting

In this section we show how efficient constructions of spanners and emulators for an unweighted graph G in the streaming setting can be used for efficient computation of approximate shortest paths.

First, one can use $O(n^\rho/\rho \cdot \log n \cdot \beta)$ passes and expected space $O(n^{1+1/\kappa} + n \cdot \log n)$ to construct an $(1 + \epsilon, \beta)$ -emulator H with $\beta = (O(\log \kappa + 1/\rho)/\epsilon)^{\log \kappa + 1/\rho}$. One can now compute $V \times V$ $(1 + \epsilon, \beta)$ -approximate shortest distances in G by computing exact $V \times V$ shortest distances in H , using the same space and without additional passes. (Observe that we do not store the output, as its size is larger than the size of H .) In particular, one can use here space of $O(n \cdot \log n)$, and have $\beta = (O(\log \log n + 1/\rho)/\epsilon)^{\log \log n + 1/\rho}$. It is also possible to set here $\rho = \frac{1}{\log \log n}$, and obtain $2^{O(\frac{\log n}{\log \log n})}$ passes, $\beta = O(\frac{\log \log n}{\epsilon})^{2 \log \log n}$.

Another option is to use space $O(n^{1+\rho} \cdot \log n)$, whp, and $O(\beta)$ passes for constructing the same emulator H . Again given the emulator we can compute $V \times V$ approximate shortest distances in G by computing shortest distances in H offline.

Corollary 20. *For any n and any parameters $0 < \epsilon \leq 1$, $2 \leq \kappa \leq (\log n)/4$, $1/\kappa \leq \rho \leq 1/2$, and any n -vertex unweighted graph G , our streaming algorithm computes $(1 + \epsilon, \beta)$ -approximate shortest distances for $V \times V$ with β given by (13). It uses in expectation either $O(n^\rho/\rho \cdot \log n \cdot \beta)$ passes and expected space $O(n^{1+1/\kappa} + n \cdot \log n)$ or $O(\beta)$ passes and space $O(n^{1+\rho} \cdot \log n)$, whp.*

If the actual paths rather than just distances are needed, then we compute a $(1 + \epsilon, \beta)$ -spanner H with β given by (14) and with expected $O(n^{1+1/\kappa})$ edges (with the restriction on κ as above). Then we compute $V \times V$ $(1 + \epsilon, \beta)$ -approximate shortest paths in H offline, using space $O(|H|)$.

Corollary 21. *For any $n, \kappa, \epsilon, \rho$ and G as in Corollary 19, a variant of our streaming algorithm computes $(1 + \epsilon, \beta)$ -approximate shortest paths for $V \times V$ with β given by (14). It uses either $O(n^\rho/\rho \cdot \log n \cdot \beta)$ passes and expected space $O(n^{1+1/\kappa} + n \cdot \log n)$ or $O(\beta)$ passes and space $O(n^{1+\rho} \cdot \log n)$, whp.*

If one is interested only in $S \times V$ paths or distances, then it is possible to eliminate the additive term of β by using $O(|S| \cdot \beta/\epsilon)$ additional passes. These passes are used to compute exactly distances between pairs $(s, v) \in S \times V$, with $d_G(s, v) \leq \beta/\epsilon$. The overall number of passes becomes $O(|S| \cdot \beta/\epsilon + n^\rho/\rho \cdot \log n \cdot \beta)$, and space $O(n^{1+1/\kappa} + n \cdot \log n)$. Whenever $|S| \geq n^{1/\kappa} \log n$, we can set $\rho = \frac{\log |S|}{\log n} - \frac{\log \log n}{\log n}$, and obtain the following corollary.

Corollary 22. *For any $n, \epsilon, \rho, \kappa$ and G as in Corollary 20, and any set $S \subseteq V$ of size at least $|S| \geq n^{1/\kappa} \log n$, a variant of our streaming algorithm computes $(1 + \epsilon)$ -approximate shortest distances for $S \times V$ in expected*

$$\frac{|S|}{\epsilon} \cdot O\left(\frac{1}{\epsilon} \left(\log \kappa + \frac{\log n}{\log |S| - \log \log n}\right)\right)^{\log \kappa + \frac{\log n}{\log |S| - \log \log n}} \quad (15)$$

passes, and space $O(n^{1+1/\kappa} + n \cdot \log n)$. To compute actual paths we have similar complexities¹⁰, but one needs to restrict κ as in Corollary 19.

Observe that for a constant $\epsilon > 0$ and $|S| = n^{\Omega(1)}$, we can take a constant κ , so the number of passes is $O(|S|)$. One can also get for $|S| = 2^{\Omega(\log n / \log \log n)}$, a streaming algorithm for computing $(1 + \epsilon)$ -approximate shortest paths for $S \times V$ by setting $\kappa = c \log n / \log \log n$, and obtaining $O(|S|/\epsilon) \cdot O(\frac{\log \log n}{\epsilon})^{2 \log \log n}$ passes and space $O(n \log n)$.

¹⁰Though there is an additional additive term of $\log^{(3)} \kappa$ in the exponent in (15).

References

- [AB16] Amir Abboud and Greg Bodwin. The $4/3$ additive spanner exponent is tight. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2016, Cambridge, MA, USA, June 18-21, 2016*, pages 351–361, 2016.
- [ABCP93] Baruch Awerbuch, Bonnie Berger, Lenore Cowen, and David Peleg. Near-linear cost sequential and distributed constructions of sparse neighborhood covers. In *34th Annual Symposium on Foundations of Computer Science, Palo Alto, California, USA, 3-5 November 1993*, pages 638–647, 1993.
- [ABN11] Ittai Abraham, Yair Bartal, and Ofer Neiman. Advances in metric embedding theory. *Advances in Mathematics*, 228(6):3026 – 3126, 2011.
- [ABP17] Amir Abboud, Greg Bodwin, and Seth Pettie. A hierarchy of lower bounds for sublinear additive spanners. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 568–576, 2017.
- [ACIM99] Donald Aingworth, Chandra Chekuri, Piotr Indyk, and Rajeev Motwani. Fast estimation of diameter and shortest paths (without matrix multiplication). *SIAM J. Comput.*, 28(4):1167–1181, 1999.
- [ADD⁺93] I. Althöfer, G. Das, D. Dobkin, D. Joseph, and J. Soares. On sparse spanners of weighted graphs. *Discrete Comput. Geom.*, 9:81–100, 1993.
- [ADF⁺17] Stephen Alstrup, Søren Dahlgaard, Arnold Filtser, Morten Stockel, and Christian Wulff-Nilsen. Personal communication, 2017.
- [AHL02] Noga Alon, Shlomo Hoory, and Nathan Linial. The moore bound for irregular graphs. *Graphs and Combinatorics*, 18(1):53–57, 2002.
- [AN12] Ittai Abraham and Ofer Neiman. Using petal-decompositions to build a low stretch spanning tree. In *STOC*, pages 395–406, 2012.
- [AP92] B. Awerbuch and D. Peleg. Routing with polynomial communication-space tradeoff. *SIAM J. Discrete Mathematics*, 5:151–162, 1992.
- [Awe85] B. Awerbuch. Complexity of network synchronization. *J. ACM*, 4:804–823, 1985.
- [Bar96] Yair Bartal. Probabilistic approximations of metric spaces and its algorithmic applications. In *FOCS*, pages 184–193, 1996.
- [Bar98] Yair Bartal. On approximating arbitrary metrics by tree metrics. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing, STOC '98*, pages 161–168, New York, NY, USA, 1998. ACM.
- [Bar04] Yair Bartal. Graph decomposition lemmas and their role in metric embedding methods. In *Algorithms - ESA 2004, 12th Annual European Symposium, Bergen, Norway, September 14-17, 2004, Proceedings*, pages 89–97, 2004.

- [Bas08] S. Baswana. Streaming algorithm for graph spanners - single pass and constant processing time per edge. *Inf. Process. Lett.*, 106(3):110–114, 2008.
- [BCE05] Béla Bollobás, Don Coppersmith, and Michael Elkin. Sparse distance preservers and additive spanners. *SIAM J. Discrete Math.*, 19(4):1029–1055, 2005.
- [BGK⁺14] Guy E. Blelloch, Anupam Gupta, Ioannis Koutis, Gary L. Miller, Richard Peng, and Kanat Tangwongsan. Nearly-linear work parallel SDD solvers, low-diameter decomposition, and low-stretch subgraphs. *Theor. Comp. Sys.*, 55(3):521–554, October 2014.
- [BKMP10] Surender Baswana, Telikepalli Kavitha, Kurt Mehlhorn, and Seth Pettie. Additive spanners and (alpha, beta)-spanners. *ACM Transactions on Algorithms*, 7(1):5, 2010.
- [BR10] Ajesh Babu and Jaikumar Radhakrishnan. An entropy based proof of the moore bound for irregular graphs. *CoRR*, abs/1011.1058, 2010.
- [BS03] S. Baswana and S. Sen. A simple linear time algorithm for computing a $(2k - 1)$ -spanner of $O(n^{1+1/k})$ size in weighted graphs. In *Proceedings of the 30th International Colloquium on Automata, Languages and Programming*, volume 2719 of *LNCS*, pages 384–396. Springer, 2003.
- [BS07] Surender Baswana and Sandeep Sen. A simple and linear time randomized algorithm for computing sparse spanners in weighted graphs. *Random Struct. Algorithms*, 30(4):532–563, 2007.
- [BW15] Gregory Bodwin and Virginia Vassilevska Williams. Very sparse additive spanners and emulators. In *Proceedings of the 2015 Conference on Innovations in Theoretical Computer Science, ITCS 2015, Rehovot, Israel, January 11-13, 2015*, pages 377–382, 2015.
- [Che13] Shiri Chechik. New additive spanners. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '13*, pages 498–512, Philadelphia, PA, USA, 2013. Society for Industrial and Applied Mathematics.
- [Coh99] E. Cohen. Fast algorithms for t -spanners and stretch- t paths. *SIAM J. Comput.*, 28:210–236, 1999.
- [Coh00] Edith Cohen. Polylog-time and near-linear work approximation scheme for undirected shortest paths. *J. ACM*, 47(1):132–166, 2000.
- [DGPV08] Bilel Derbel, Cyril Gavoille, David Peleg, and Laurent Viennot. On the locality of distributed sparse spanner construction. In *Proceedings of the Twenty-Seventh Annual ACM Symposium on Principles of Distributed Computing, PODC 2008, Toronto, Canada, August 18-21, 2008*, pages 273–282, 2008.
- [DHZ00] D. Dor, S. Halperin, and U. Zwick. All-pairs almost shortest paths. *SIAM J. Comput.*, 29:1740–1759, 2000.
- [DMP⁺03] D. Dubhashi, A. Mei, A. Panconesi, J. Radhakrishnan, and A. Srinivisan. Fast distributed algorithm for (weakly) connected dominating sets and linear-size skeletons, 2003.

- [DMZ06] Bilel Derbel, Mohamed Mosbah, and Akka Zemmari. Fast distributed graph partition and application. In *20th International Parallel and Distributed Processing Symposium (IPDPS 2006), Proceedings, 25-29 April 2006, Rhodes Island, Greece, 2006*.
- [EEST05] Michael Elkin, Yuval Emek, Daniel A. Spielman, and Shang-Hua Teng. Lower-stretch spanning trees. In *STOC*, pages 494–503, 2005.
- [Elk04] M. Elkin. An unconditional lower bound on the time-approximation tradeoff of the minimum spanning tree problem. In *Proc. of the 36th ACM Symp. on Theory of Comput. (STOC 2004)*, pages 331–340, 2004.
- [Elk05] Michael Elkin. Computing almost shortest paths. *ACM Trans. Algorithms*, 1(2):283–323, 2005.
- [Elk07a] Michael Elkin. A near-optimal distributed fully dynamic algorithm for maintaining sparse spanners. In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing, PODC 2007, Portland, Oregon, USA, August 12-15, 2007*, pages 185–194, 2007.
- [Elk07b] Michael Elkin. Streaming and fully dynamic centralized algorithms for constructing and maintaining sparse spanners. In *Automata, Languages and Programming, 34th International Colloquium, ICALP 2007, Wroclaw, Poland, July 9-13, 2007, Proceedings*, pages 716–727, 2007.
- [EN16a] Michael Elkin and Ofer Neiman. Distributed strong diameter network decomposition. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing, PODC 2016, Chicago, IL, USA, July 25-28, 2016*, pages 211–216, 2016.
- [EN16b] Michael Elkin and Ofer Neiman. Hopsets with constant hopbound, and applications to approximate shortest paths. In *IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA*, pages 128–137, 2016.
- [EN17] Michael Elkin and Ofer Neiman. Efficient algorithms for constructing very sparse spanners and emulators. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 652–669, 2017.
- [EP04] Michael Elkin and David Peleg. $(1+\epsilon, \beta)$ -spanner constructions for general graphs. *SIAM J. Comput.*, 33(3):608–631, 2004.
- [EP15] Michael Elkin and Seth Pettie. A linear-size logarithmic stretch path-reporting distance oracle for general graphs. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2015, San Diego, CA, USA, January 4-6, 2015*, pages 805–821, 2015.
- [ES16] Michael Elkin and Shay Solomon. Fast constructions of lightweight spanners for general graphs. *ACM Trans. Algorithms*, 12(3):29:1–29:21, 2016.
- [EZ06] Michael Elkin and Jian Zhang. Efficient algorithms for constructing $(1+\epsilon, \beta)$ -spanners in the distributed and streaming models. *Distributed Computing*, 18(5):375–385, 2006.

- [FKM⁺05] J. Feigenbaum, S. Kannan, A. McGregor, S. Suri, and J. Zhang. Graph distances in the streaming model: The value of space. In *Proc. of the ACM-SIAM Symp. on Discrete Algorithms*, pages 745–754, 2005.
- [FS16] Arnold Filtser and Shay Solomon. The greedy spanner is existentially optimal. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing, PODC 2016, Chicago, IL, USA, July 25-28, 2016*, pages 9–17, 2016.
- [HZ96] S. Halperin and U. Zwick. Linear time deterministic algorithm for computing spanners for unweighted graphs, 1996. manuscript.
- [KMP11] Ioannis Koutis, Gary L. Miller, and Richard Peng. A nearly- $m \log n$ time solver for sdd linear systems. In *Proceedings of the 2011 IEEE 52Nd Annual Symposium on Foundations of Computer Science, FOCS '11*, pages 590–598, Washington, DC, USA, 2011. IEEE Computer Society.
- [LS93] N. Linial and M. Saks. Decomposing graphs into regions of small diameter. *Combinatorica*, 13:441–454, 1993.
- [MPVX15] Gary L. Miller, Richard Peng, Adrian Vladu, and Shen Chen Xu. Improved parallel algorithms for spanners and hopsets. In *Proceedings of the 27th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '15*, pages 192–201, New York, NY, USA, 2015. ACM.
- [MPX13] Gary L. Miller, Richard Peng, and Shen Chen Xu. Parallel graph decompositions using random shifts. In *25th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '13, Montreal, QC, Canada - July 23 - 25, 2013*, pages 196–203, 2013.
- [OMSW10] James B. Orlin, Kamesh Madduri, K. Subramani, and M. Williamson. A faster algorithm for the single source shortest path problem with few distinct positive lengths. *J. of Discrete Algorithms*, 8:189–198, June 2010.
- [Pel99] David Peleg. Proximity-preserving labeling schemes and their applications. In *Graph-Theoretic Concepts in Computer Science, 25th International Workshop, WG '99, Ascona, Switzerland, June 17-19, 1999, Proceedings*, pages 30–41, 1999.
- [Pel00] David Peleg. *Distributed Computing: A Locality-sensitive Approach*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2000.
- [Pet09] Seth Pettie. Low distortion spanners. *ACM Transactions on Algorithms*, 6(1), 2009.
- [Pet10] Seth Pettie. Distributed algorithms for ultrasparse spanners and linear size skeletons. *Distributed Computing*, 22(3):147–166, 2010.
- [Pet16] Seth Pettie. Personal communication, 2016.
- [PS89] D. Peleg and A. Schäffer. Graph spanners. *J. Graph Theory*, 13:99–116, 1989.
- [PU89a] D. Peleg and J. D. Ullman. An optimal synchronizer for the hypercube. *SIAM J. on Comput.*, 18:740–747, 1989.

- [PU89b] D. Peleg and E. Upfal. A tradeoff between size and efficiency for routing tables. *J. of the ACM*, 36:510–530, 1989.
- [Rei93] John H. Reif. *Synthesis of Parallel Algorithms*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1st edition, 1993.
- [RZ04] Liam Roditty and Uri Zwick. On dynamic shortest paths problems. In *Algorithms - ESA 2004, 12th Annual European Symposium, Bergen, Norway, September 14-17, 2004, Proceedings*, pages 580–591, 2004.
- [TZ05] Mikkel Thorup and Uri Zwick. Approximate distance oracles. *J. ACM*, 52(1):1–24, 2005.
- [TZ06] M. Thorup and U. Zwick. Spanners and emulators with sublinear distance errors. In *Proc. of Symp. on Discr. Algorithms*, pages 802–809, 2006.
- [Woo06] David P. Woodruff. Lower bounds for additive spanners, emulators, and more. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science, FOCS '06*, pages 389–398, Washington, DC, USA, 2006. IEEE Computer Society.